Fraus

Medição de satisfação sem LLM em runtime

João Pedro Tavares Vicente

Table of contents

1. O cliente *diz* uma coisa. O texto mede outra. 8
1.1 O que ler primeiro 9
1.2 Sem LLM em runtime 11
1.3 Baixar 11
2. A tese 12
2.1 O problema 12
2.2 A aposta 12
2.3 O que sai disso, e que nada mais consegue calcular 12
2.4 O que a tese NÃO promete 12
3. As dez invariantes 14
3.1 1. Sem LLM em runtime 14
3.2 2. Ausência de dado não é insatisfação 14
3.3 3. Score, nota e categoria são derivados no servidor 14
3.4 4. Faixas de NPS: 0–6 detrator, 7–8 neutro, 9–10 promotor 15
3.5 5. Latência nunca é persistida 15
3.6 6. Timestamps timezone-aware 15
3.7 7. Modelo ausente é falha alta e explícita 16
3.8 8. A ordem das classes é 0 insatisfeito, 1 neutro, 2 satisfeito 16
3.9 9. As 39 chaves de feature batem exatamente com `NOMES_FEATURES` 16
3.10 10. O corpus de treino não pode entregar o rótulo 17
4. Arquitetura 18
4.1 A decisão que explica o desenho 18
4.2 As sete famílias que pontuam 19
4.3 Onde roda, e com que motor 19
4.4 A ironia continua carregada, e não pontua 20
4.5 Régua: o que pontuou uma conversa não é fixo no tempo 20
4.6 Repontuar é um trabalho, não uma requisição 20
4.7 A explicação da nota 21
4.8 Confiança: conjunto conformal e abstenção 21
4.9 Sinalizadores por regra 22
4.10 Impacto por tema e deterioração 22
4.11 Onde cada coisa mora 22
5. Treinamento dos modelos do Fraus 24
5.1 Como abrir no Colab 26

5.2 Notebook 01 — classificador de texto 26
5.2.1 Exigencia de GPU 26
5.2.2 Onde o checkpoint fica no Drive 27
5.2.3 Para onde copiar o artefato no repo local 27
5.2.4 Mapeamento de rotulo 27
5.3 Notebook 02 — fusor dos tres sinais 28
5.3.1 Rotulo do fusor: `recommend_to_a_friend` 28
5.3.2 Limitacao metodologica a declarar no relatorio 29
5.3.3 O vazamento do primeiro fusor — 13/08/2026 29
5.3.4 O fusor de 35 features — 24/08/2026 30
5.3.5 Consequencia no NPS: a classe neutra cai em detrator — HISTORICO (fusor de 16 features, ate 24/08/2026) 31
5.3.6 Retreino do fusor apos as features de incongruencia (03/09/2026) 32
5.3.7 A ironia sai do vetor (04/09/2026) 34
5.3.8 O fusor de 38 features — 04/09/2026 35
5.3.9 O contrato sobe para 39: incongruencia implicita (04/09/2026) 36
5.3.10 Conferencia do artefato 36
5.4 Contrato de features 37
5.5 Notebook 03 — classificador de emocao 38
5.5.1 Ordem canonica das classes 38
5.5.2 Corpus: `go_emotions_ptbr` 38
5.5.3 O desequilibrio, e por que NAO subamostrar 39
5.5.4 O teto do desprezo — a declarar no relatorio 39
5.5.5 Por que o desprezo NAO e treinado 39
5.6 Notebook 04 — classificador de ironia 40
5.6.1 Fontes publicas do treino e teste oficial fechado 40
5.6.2 A divisao por autor — conserto de 02/10/2026 41
5.6.3 Candidato Laya 42
5.6.4 Alternativas ao IDPT — levantamento de 14/08/2026 43
5.6.5 Onde existe corpus PT-BR de ironia (e como pedir) 43
5.6.6 O padrao: corpus sintetico 43
5.6.7 Portao de promocao do artefato 44
5.7 Sinal lexico — SentiLex-PT02 45
5.8 Sinal de estilo 45
5.9 Calibração — a temperatura mora no artefato (desde 19/10/2026) 46
5.10 Notebook 02 — a seção 10 é uma pendência, não uma etapa 47
5.10.1 As três saídas, medidas em 15/09/2026 — decidido: só `log1p` 47
5.10.2 Promovido em 15/09/2026 48

6. Ironia: corpus e Laya 49
6.1 Resumo: consertar o corpus antes de treinar o Laya 49
6.2 O IDPT entrega o rótulo, e nenhum encoder escapa disso 49
6.3 Treinar só ironia exige mexer em sete células e num rótulo de produção 50
6.4 Consertar o negativo vale mais que trocar o encoder 53
6.5 Três treinos e uma régua de 300 pares separam as causas 53
6.6 O ganho esperado é de poucos pontos, e vem do dado 55
6.7 Régua de pares mínimos 56
6.8 Conclusão 56
7. Integração com o Tars 58
7.1 Conectar, em dois passos 58
7.2 O que é um atendimento 58
7.3 Quando o envio acontece 58
7.4 O pedido 59
7.5 Privacidade — o que vale e o que falta 59
7.6 Limpeza 60
8. Design da interface — Contexto, não certeza 61
8.1 Direção 61
8.1.1 Camadas do FR 61
8.2 Cor e significado 63
8.3 Tipografia e ícones 64
8.4 LP 64
8.4.1 Estrutura e conteúdo 64
8.4.2 Céu 65
8.4.3 Widgets da LP 65
8.4.4 Tipografia da LP 65
8.4.5 Coreografia 65
8.4.6 FR e palco 66
8.5 Dashboard 66
8.5.1 Leitura progressiva — 22/10/2026 68
8.5.2 Atividade e carregamento — 22/10/2026 68
8.5.3 Números, ausência e proveniência 68
8.5.4 Encaixe secreto 68
8.5.5 Temas 69
8.6 Acesso, acessibilidade e estados 69
8.7 Verificação e manutenção 69
9. LP e dashboard — antes e depois 71
9.1 Condições da comparação 71

9.2 Landing page 72
9.2.1 O que mudou 0
9.2.2 Widgets e leitura 0
9.2.3 O fundo não termina no FR 0
9.2.4 Celular 0
9.3 Dashboard 0
9.3.1 Alternativa clara 0
9.3.2 Sidebar e navegação 0
9.3.3 Um segredo que não mexe nos dados 0
9.3.4 Celular 0
9.4 Otimização da LP aprovada 0
9.4.1 Ensaio controlado de rolagem e carga 0
9.4.2 Reproduzir a verificação funcional 0
9.5 Verificação e limites 0
10. Conformidade regulatória — o Fraus e o EU AI Act 0
10.1 1. Por que este documento existe 0
10.2 2. Escopo do sistema 0
10.3 3. As três obrigações que já valem, e a que vem 0
10.3.1 3.1 Transparência como compromisso do produto 0
10.3.2 3.2 Limite de finalidade: não pontuar trabalhadores 0
10.3.3 Regra: **o Fraus não pontua atendentes.** 0
10.3.4 3.3 Controles recomendados para uma implantação comercial 0
10.4 4. Limitações que importam para quem lê um número do Fraus 0
10.5 5. Fontes 0
11. Encolher os modelos — o que foi medido, e por que a int8 foi recusada 0
11.1 O problema 0
11.2 O que foi testado 0
11.3 O resultado da melhor variante 0
11.4 E é aqui que ela é recusada 0
11.5 A variante mista, medida em 11/09/2026 0
11.5.1 De quem é a culpa, então 0
11.5.2 E o que isso faz com a máquina de 1 GB 0
11.5.3 O que sobra de ganho real 0
11.6 O backend ONNX em produção, medido em 15/09/2026 0
11.6.1 Emoção com int8 só nos pesos — aceita pelo portão, não promovida por padrão 0
11.7 O que sobra como caminho 0
11.8 Reproduzir 0
11.9 Variante experimental: um encoder, três cabeças 0

12. Limitações conhecidas 0
12.1 Limitações operacionais do deploy gratuito 0
12.2 O NPS é inferido, não perguntado 0
12.2.1 Calibração e conjunto conformal do fusor (E2) 0
12.2.2 Sinalizadores por regra (E5) 0
12.2.3 Impacto por tema e deterioração (E6) 0
12.3 O sinal de tempo é treinado em dados sintéticos 0
12.4 A cabeça de ironia é pouco confiável 0
12.5 Desprezo não é uma classe treinada 0
12.6 Nenhuma feature agregada reverte uma probabilidade saturada 0
12.7 A explicação é do modelo, não do cliente 0
12.8 O que não dá para consertar sem corpus de atendimento real 0
13. Cartão do modelo 0
13.1 Uso pretendido e uso proibido 0
13.2 As peças 0
13.3 Onde o fusor erra — avaliação por fatia 0
13.4 Conversa só de cortesia — sem sinal desde 15/09/2026 0
13.5 A cabeça de ironia — taxa de erro publicada 0
13.6 Calibração e conformal do fusor 0
13.7 Limitações que valem para tudo acima 0
14. Diferenciais 0
14.1 Diferenciais — relatório de entrega (14/10/2026) 0
14.1.1 Resumo 0
14.1.2 O que vale para todas as entregas 0
14.1.3 E1 — "Por que esta nota", contrafactual de espera e drivers 0
14.1.4 E2 — Calibração, abstenção e conjunto conformal 0
14.1.5 E5 — Sinalizadores por regra 0
14.1.6 E6 — Impacto por tema e início da deterioração 0
14.1.7 Contrato da API 0
14.1.8 Verificação final 0
14.1.9 Reproduzir 0
14.1.10 Pendências que dependem do João 0
14.2 Diferenciais — o que os concorrentes fazem (fontes de 14/10/2026) 0
14.2.1 Leitura honesta do resultado 0
14.2.2 Intercom — Fin CX Score 0
14.2.3 Zendesk — Zendesk QA (ex-Klaus) / AutoQA 0
14.2.4 Observe.AI 0
14.2.5 Medallia — Conversational Intelligence 0

14.2.6 Zenvia (incorporou a Movidesk) 0
14.2.7 Blip (Take Blip) 0
14.2.8 O que NÃO pode ser afirmado 0
14.3 Diferenciais — roteiro da demonstração 0
14.3.1 Antes de começar 0
14.3.2 1. A tese (1 min) 0
14.3.3 2. Um atendimento, de ponta a ponta (4 min) 0
14.3.4 3. O agregado (2 min) 0
14.3.5 4. Retrospectiva de uma conversa (1 min) 0
14.3.6 5. O que o Fraus não faz (slide, 1 min) 0
14.3.7 6. Perguntas prováveis da banca 0
15. Referência de código 0
15.1 Indicadores 0
15.1.1 fraus.indicadores 0
15.2 Modelo canônico 0
15.2.1 fraus.modelos 0
15.3 O fusor e as 39 features 0
15.3.1 fraus.fusor 0
15.4 Os sinais 0
15.4.1 Texto — o único que pontua sozinho 0
15.4.2 fraus.sinais.texto 0
15.4.3 Emoção 0
15.4.4 fraus.sinais.emocao 0
15.4.5 Tempo 0
15.4.6 fraus.sinais.tempo 0
15.4.7 Léxico 0
15.4.8 fraus.sinais.lexico 0
15.5 Evidência e deriva 0
15.5.1 Evidência fraca — a cabeça tracejada 0
15.5.2 fraus.evidencia 0
15.5.3 Deriva de distribuição — a invariante 10 em runtime 0
15.5.4 fraus.deriva 0

FRAUS — DOCUMENTAÇÃO

1. O cliente diz uma coisa. O texto mede outra.

Ferramenta que lê atendimento por chatbot e estima satisfação **sem perguntar nada ao cliente**. O caso que uma pesquisa de NPS declarada não captura: quem escreve "*ok, obrigado* 🙂" e sai insatisfeito não responde pesquisa — e é ele que o sistema precisa enxergar.

LER AS INVARIANTES | A TESE | ARQUITETURA

FAMÍLIAS DE SINAL	FEATURES NO FUSOR	BERTIMBAU FINE-TUNADOS	LLMS NA INFERÊNCIA
07	39	03	00

⚠ TODO NÚMERO AQUI É INFERIDO

O NPS do Fraus **não é perguntado**: ele é derivado do texto do atendimento. Toda exibição carrega a etiqueta de estimativa, e apresentá-lo como NPS declarado seria falso. Ver Limitações e Conformidade.

1.1 O QUE LER PRIMEIRO

A TESE

A aposta do projeto e o caso que ela cobre.

AS INVARIANTES

Para mexer no código: comece por aqui, sem exceção.

ARQUITETURA

Como as peças se ligam, do adaptador de ingestão ao dashboard.

TREINAMENTO

Como os modelos foram treinados e o que foi medido.

DESIGN DA INTERFACE

FR em camadas, tipografia quadrada, temas e a distinção dito/medido.

ANTES E DEPOIS DA INTERFACE

LP e dashboard comparadas em desktop e celular, com limites da verificação.

LIMITAÇÕES

O que o sistema **não** sabe, dito sem rodeio.

CONFORMIDADE

O que o produto lê e o que ele se recusa a fazer.

ENCOLHER OS MODELOS

Como o runtime ficou pequeno sem trocar o modelo, medido.

CARTÃO DO MODELO

Desempenho por fatia, sem esconder a fatia ruim.

1.2 SEM LLM EM RUNTIME

Três BERTimbau fine-tunados (satisfação, emoção, ironia) e um fusor `LogisticRegression` sobre 39 features de sete famílias de sinal. Inferência local, em CPU, sem nenhuma chamada de rede no caminho de predição.

Isso é **requisito**, não otimização: o trabalho existe para demonstrar que dá para medir isso com modelos pequenos e interpretáveis — coeficiente de regressão logística responde *por que* um atendimento recebeu a nota, e floresta densa não.

1.3 BAIXAR

O PDF com o site inteiro está em pdf/fraus.pdf, gerado no mesmo build que publica estas páginas — então ele nunca está mais velho que o site.

2. A tese

2.1 O PROBLEMA

Quem opera um chatbot de atendimento mede satisfação de dois jeitos, e os dois falham no mesmo ponto.

Pesquisa declarada (NPS, CSAT). Depende de resposta, e a taxa de resposta é baixa e enviesada: responde quem ficou muito irritado ou muito satisfeito. O cliente do meio — o que resolveu o problema com esforço, ou desistiu no meio — simplesmente não aparece.

Taxa de contenção. Mede se o bot segurou o atendimento sem escalar. É um número de **volume**, não de qualidade: sobe igual quando o bot resolve e quando o cliente desiste de tentar.

Cruzados, os dois produzem o pior cenário possível: contenção alta com satisfação não medida. O painel fica verde e ninguém sabe se a operação está boa.

2.2 A APOSTA

O texto do atendimento **já contém** a evidência da satisfação. Ela não precisa ser perguntada — precisa ser lida.

O caso canônico do projeto é este:

> "*que atendimento maravilhoso, só esperei 3 horas*"

Uma pesquisa nunca vai receber essa frase. Um classificador de sentimento ingênuo lê "maravilhoso" e marca promotor. O sistema precisa das duas metades ao mesmo tempo: o que foi **dito** e o que foi **medido**.

2.3 O QUE SAI DISSO, E QUE NADA MAIS CONSEGUE CALCULAR

Falso containment. O cruzamento de "não escalou" com "saiu detrator". É a tese virando número, e só quem infere satisfação do texto consegue calcular: quem depende de pesquisa respondida perde justamente o cliente contido, que é o que não responde.

2.4 O QUE A TESE NÃO PROMETE

- **Não é NPS declarado.** É estimativa, e a interface diz isso em todo lugar.
- **Não decide nada sozinha.** Nenhuma saída aciona ação sobre ninguém.
- **Não pontua atendentes** — ver Conformidade.

- **Não é infalível, e diz onde falha.** O sinal de tempo é treinado em dados sintéticos; a cabeça de ironia é pouco confiável e por isso não pontua; a espera entra em `log1p` desde 15/09/2026 e pesa cada vez mais devagar (3 h ≈ 2,1× o texto), mas fora da faixa do treino continua fora da distribuição, e `GET /saude/deriva` mostra isso. Tudo isso está em Limitações, com número.

3. As dez invariantes

Não são preferências de estilo. **Cada uma nasceu de um defeito real**, e violá-la é reintroduzir o defeito — por isso cada regra aqui vem com a cicatriz que a produziu. Documentação que explica cicatriz é a que não dá para copiar de tutorial.

O formato é sempre o mesmo: **a regra**, *por que ela existe*, **o que quebra se alguém a violar**, e onde ela mora no código.

3.1 1. SEM LLM EM RUNTIME

A regra. Inferência local em CPU. Nenhuma chamada de rede no caminho de predição.

Por que existe. É requisito do trabalho, não otimização de custo. A tese é que dá para medir satisfação com modelos pequenos, fine-tunados e interpretáveis — chamar uma API de LLM não deixaria o sistema mais rápido, e invalidaria o que ele se propõe a demonstrar.

O que quebra. O trabalho inteiro. Nenhum teste pega isso: o código funcionaria melhor e a defesa cairia.

Onde mora. Três BERTimbau em `fraus/sinais/` e um `LogisticRegression` em `fraus/fusor.py`.

3.2 2. AUSÊNCIA DE DADO NÃO É INSATISFAÇÃO

A regra. Conversa sem fala do cliente tem `score: None` e aparece como "sem sinal" — nunca `0`, em lugar nenhum: célula, gráfico, ordenação, export, agregado.

Por que existe. `None` e `0` significam coisas diferentes e não podem se confundir. Zero numa coluna de tempo de resposta se lê como "respondeu na hora", e a conversa que nunca teve atendente humano apareceria como **a mais ágil da operação**. Zero num NPS se lê como um número que alguém mediu.

O que quebra. Silenciosamente, tudo: o pior valor da escala aparece onde a verdade era "não medi". Em 10/09/2026 essa regra ainda estava furada dentro do próprio arquivo que mais a defende — `containment_rate` devolvia `0.0` no conjunto vazio, e o front tinha uma guarda para não acreditar nele.

Onde mora. Em toda parte. Procure `?? 0` e `|| 0` antes de commitar — e **também** `return 0` **no Python**, que é o mesmo defeito disfarçado de inicialização.

3.3 3. SCORE, NOTA E CATEGORIA SÃO DERIVADOS NO SERVIDOR

A regra. Nunca aceitos do corpo da requisição, nunca recalculados no cliente.

Por que existe. O `round` do Python é bancário (`round(6.5) == 6`) e o `Math.round` do JavaScript arredonda meio para cima (`Math.round(6.5) == 7`).

O que quebra. Já quebrou: a tabela exibia **nota 7 ao lado da etiqueta "Detrator"**, nas fronteiras 6/7 e 8/9. Duas fontes da verdade para o mesmo número, discordando na tela do usuário.

Onde mora. `fraus/indicadores.py::nota_0_10`. O front consome; não recalcula.

3.4 4. FAIXAS DE NPS: 0–6 DETRATOR, 7–8 NEUTRO, 9–10 PROMOTOR

A regra. Padrão de fábrica em `FAIXAS_NPS`, configurável por `PUT /configuracoes` — e só se as faixas cobrirem 0..10 de forma contígua. A faixa vigente é passada **por parâmetro**, nunca lida de estado global.

Por que existe. Estado global mutável muda sob os pés de quem já calculou. E faixa com buraco ou sobreposição não é preferência de gosto: é nota sem categoria, ou com duas.

O que quebra. Já quebrou: `categoriaDaNota` no front tinha `<= 6` e `<= 8` digitados no corpo, e isso rodava no caminho feliz. Um operador que mexesse em Configurações via a **mesma tela dar dois vereditos** para o mesmo atendimento — a etiqueta da linha com as faixas novas, a cor da barra com as antigas.

Onde mora. `fraus/indicadores.py::FAIXAS_NPS`, `categoria_nps`, `validar_faixas_nps`.

3.5 5. LATÊNCIA NUNCA É PERSISTIDA

A regra. Sempre derivada dos timestamps, na leitura.

Por que existe. Latência gravada é latência que envelhece: qualquer correção na definição de "espera" deixaria o banco cheio de números calculados pela regra antiga, indistinguíveis dos novos.

O que quebra. Duas definições de latência convivendo — uma para a lista, outra para o detalhe — e as duas telas discordando sobre o mesmo atendimento.

Onde mora. `fraus/sinais/tempo.py::latencias_da_conversa`.

3.6 6. TIMESTAMPS TIMEZONE-AWARE

A regra. `datetime` naive é erro de validação, não conversão silenciosa.

Por que existe. Data sem fuso é data que muda de significado conforme quem lê.

O que quebra. A armadilha já paga: **a data da Totalk é `MM/DD/YYYY`**, não `DD/MM`. Lida como brasileira, espalha as mensagens por cinco meses e a latência sai absurda **em silêncio**.

Onde mora. `fraus/modelos.py`.

3.7 7. MODELO AUSENTE É FALHA ALTA E EXPLÍCITA

A regra. Sem fallback, sem motor dublê silencioso. Nenhuma predição sai sem os três modelos treinados e o fusor. A API sobe sem carregá-los (o `ProvedorDeMotor`, em `fraus/api/motor_preguicoso.py`, carrega na primeira rota que exige o motor), mas a falha de carga é memorizada e sobe de novo em toda chamada, e `GET /saude/prontidao` responde 503 enquanto o motor não está `pronto`. Não existe queda para dublê.

Por que existe. Servir predição sem modelo carregado é **pior que estar fora do ar**: a interface não distingue número real de número sintético.

O que quebra. Já quebrou, e custou um dia: o `scripts/api_demo.py` estava no ar por causa do diretório de onde foi lançado, e a dashboard escreveu "API no ar" durante um dia inteiro enquanto **todo número da tela era sintético** — os pesos por feature saíam numa progressão 0,2 / 0,25 / 0,3 e passavam por coeficiente de regressão logística.

Onde mora. `Fusor.carregar` (valida `n_features_in_`), `ProvedorDeMotor` (`fraus/api/motor_preguicoso.py`), `GET /saude` (`"motor": "real" | "duble"` e `"estado_motor"`), `GET /saude/prontidao`, e `GET /saude/deriva`, que recusa diagnosticar com dublê pela mesma lógica.

3.8 8. A ORDEM DAS CLASSES É 0 INSATISFEITO, 1 NEUTRO, 2 SATISFEITO

A regra. No notebook, no sinal de texto, no fusor e nos indicadores.

Por que existe. É a única convenção que amarra quatro lugares que não se importam por código.

O que quebra. Nada estoura. O sistema pontua ao contrário, em silêncio, com a mesma confiança de sempre. É a invariante mais barata de violar e a mais cara de descobrir.

Onde mora. `fraus/sinais/texto.py` (`INSATISFEITO`, `NEUTRO`, `SATISFEITO`).

3.9 9. AS 39 CHAVES DE FEATURE BATEM EXATAMENTE COM `NOMES_FEATURES`

A regra. As sete famílias do vetor produzem exatamente 39 chaves. `vetorizar` levanta `KeyError` na falta — nunca zero silencioso.

Por que existe. Zero numa feature ausente não é neutro: o modelo aprendeu o que zero significa naquela dimensão, e a conversa entraria como se tivesse sido medida.

O que quebra. Já quebrou de outro jeito: o `CLAUDE.md` chegou a declarar "38" na invariante e "(35)" no mapa — dois números de duas revisões, nenhum o vigente. A documentação que governa toda sessão de agente envelhecendo em silêncio. Hoje há teste amarrando o número do `CLAUDE.md` a `len(NOMES_FEATURES)`.

Onde mora. `fraus/fusor.py::NOMES_FEATURES` — e o comentário dele conta a história inteira: 40 − 2 (a ironia saiu) + 1 (`incongruencia_situacao_negativa` entrou) = 39.

3.10 10. O CORPUS DE TREINO NÃO PODE ENTREGAR O RÓTULO

A regra. Distribuição por rótulo se sobrepõe; feature constante no treino nasce com peso zero. **Acurácia alta demais é sintoma, não vitória.**

Por que existe. Faixa de latência disjunta por classe fez o primeiro fusor marcar **99,3% lendo só o relógio**, com o BERTimbau apagado.

O que quebra. Um número excelente que não mede nada. E ela não para no treino: em 08/09/2026 a mesma invariante apareceu **em runtime** — numa conversa com 3 h de espera, a latência contribuiu −85,4 contra +4,6 do texto inteiro, e o score saiu em `4,4e-24`. O corpus nunca passou de minutos, então a guarda de vazamento, que olha o corpus, não tinha o que dizer.

Onde mora. `docs/treinamento.md`, e agora também `GET /saude/deriva`, que transforma a regra de disciplina de notebook em alarme de runtime.

4. Arquitetura



** a cabeça de ironia é lida por mensagem e mostrada na tela, mas não entra nas 39 features do Fusor — ver a seção abaixo.*

4.1 A DECISÃO QUE EXPLICA O DESENHO

Um modelo canônico, muitos adaptadores. CSV, Totalk, transcrição de `.docx`/`.pdf` e webhook desembocam todos em `Conversa` — e o resto do sistema não sabe de onde o dado veio. Adicionar uma fonte é escrever um adaptador, não tocar em sinal, fusor ou indicador.

4.2 AS SETE FAMÍLIAS QUE PONTUAM

FAMÍLIA	O QUE MEDE	POR QUE EXISTE
texto	P(insatisfeito/neutro/satisfeito) por mensagem	o sinal principal — o único fine-tunado para a tarefa
emoji	polaridade e **posição relativa**	emoji no fim da conversa não vale o mesmo que no começo
tempo	latências, escalação, abandono	espera longa degrada satisfação (literatura de live chat)
emoção	6 emoções básicas de Ekman + neutro, e desprezo derivado de raiva + nojo	raiva e frustração distinguem *tipos* de insatisfação
léxico	SentiLex-PT02 com escopo de negação	âncora determinística, auditável, imune a retreino
estilo	caixa alta, alongamento, palavrão, censura	intensidade que o classificador de sentença perde
incongruência	elogio convivendo com situação negativa	é o que alcança a frase canônica do projeto

4.3 ONDE RODA, E COM QUE MOTOR

Backend dos modelos	`FRAUS_BACKEND=torch` (padrão), `onnx` — os mesmos checkpoints, sem torch em runtime — ou `onnx-multitarefa` (candidato, não promovido); valor desconhecido é erro de boot
Cabeça de ironia	`FRAUS_IRONIA_BACKEND`: `padrao` (BERTimbau), `laya` ou `laya-onnx` — troca só a leitura por mensagem; a ironia não entra no score
Motor	carregado por um provedor preguiçoso (`ProvedorDeMotor`): `frio` → `carregando` → `pronto` ou `erro`, sem fallback para dublê
Aquecimento	a carga começa em segundo plano no boot e a cada `GET /saude`; `GET /saude/prontidao` só dá 200 com o motor pronto
Produção	dashboard e API em dois projetos Vercel, estado no Supabase Postgres; a API é serverless e **esfria**

O aquecimento reduz o cold start e não o elimina. Para eliminá-lo, a API precisa de um container sempre ligado (ver Limitações).

4.4 A IRONIA CONTINUA CARREGADA, E NÃO PONTUA

A oitava cabeça existe, é obrigatória para a API subir e aparece na tela **por mensagem** — mas **saiu do vetor** em 04/09/2026.

Medida no próprio corpus de treino do fusor (B2W-Reviews01), ela marca **74% das resenhas satisfeitas** como irônicas contra **9% das insatisfeitas**: naquele corpus ela é um detector de sentimento **positivo**, não de ironia. O fusor aprendeu peso **+0,77** para `ironia_prob_media`, empurrando ironia para *satisfeito* — o inverso do que o nome promete.

> **Feature que mede outra coisa que não o nome dela é pior que feature ausente.** Ela some do vetor sem que ninguém perceba a inversão, porque a acurácia global continua ótima.

Ver Treinamento para a medição completa.

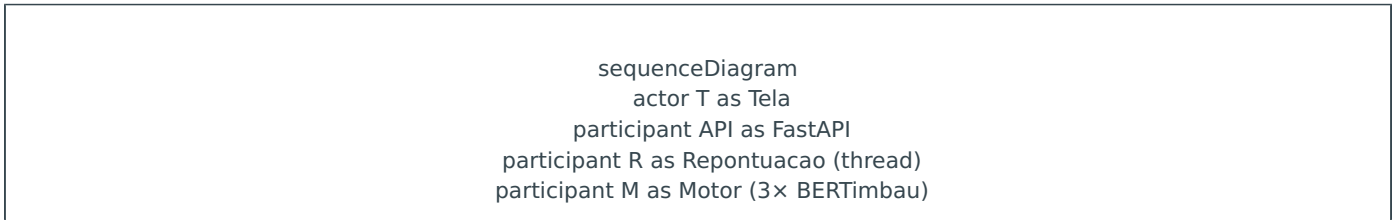
4.5 RÉGUA: O QUE PONTUOU UMA CONVERSA NÃO É FIXO NO TEMPO

Cada conversa é gravada com a **régua** que a pontuou: a assinatura do motor servindo (`Motor.regua`, `None` no motor dublê) mais a versão da curadoria (`lexico_versao`). Trocar de checkpoint, trocar de backend (torch/onnx) ou o analista ensinar um termo novo ao léxico muda a régua — e o banco passa a ter conversas antigas pontuadas por uma régua que não é mais a vigente.

O sistema não recalcula sozinho: ele **avisa**. `GET /indicadores` expõe `pontuadas_com_regua_antiga`, contando contra a régua vigente lida a cada requisição (nunca guardada em memória, pelo mesmo motivo de `faixas_vigentes` e `curadoria_vigente` — sem cópia de estado envelhecendo). A tela mostra o aviso com o motivo (checkpoint novo, léxico curado, ou os dois), nunca um número solto sem explicação.

4.6 REPONTUAR É UM TRABALHO, NÃO UMA REQUISIÇÃO

Zerar o aviso de régua misturada significa rodar os três BERTimbau de novo em cada conversa do banco — em milhares de atendimentos isso estoura qualquer timeout HTTP. `POST /conversas/repontuar` (`fraus/api/repontuacao.py`) só dispara uma thread em segundo plano; `GET /conversas/repontuar` lê o progresso (`total`, `feitas`, `estado`, `erro`) que a tela transforma em barra. Não é fila — é **um trabalho por vez**: uma segunda chamada enquanto a primeira roda recebe 409, porque duas repontuações em paralelo gravariam o mesmo banco com duas curadorias diferentes. O estado mora na memória do processo (reiniciar a API no meio perde o progresso, não o que já foi gravado — cada conversa salva já carrega sua própria `lexico_versao` e régua).



participant DB as banco

T->>API: POST /conversas/repontuar
API->>R: iniciar(ctx)
Note over R: le curadoria, faixas e régua
UMA vez — janela fechada
R-->>API: estado=rodando, total=N
API-->>T: 202 (ou 409 se já rodando)
R->>M: dispara thread em segundo plano

loop cada conversa do lote
M->>M: pontuar_conversa
M->>DB: salvar(score, lexico_versao, régua)
R->>R: feitas += 1
end

loop enquanto roda
T->>API: GET /conversas/repontuar
API-->>T: {total, feitas, estado}
Note over T: barra de progresso
end

R-->>API: estado=concluido (ou falhou)
T->>API: GET /conversas/repontuar
API-->>T: estado final

4.7 A EXPLICAÇÃO DA NOTA

`GET /conversas/{id}/atribuicao` devolve também o campo `explicacao`: a decomposição exata do fusor (intercepto e contribuições por feature, somando no logit), o efeito por família e frases geradas por template — nenhum LLM. Duas rotas completam o E1: `POST /conversas/{id}/contrafactual` refaz a nota mudando só o relógio (e recusa o que sai de `|z| ≤ 4`) e `GET /indicadores/drivers` agrega os efeitos numa amostra com semente. Nenhuma das três grava; o score, a nota e a categoria gravados não mudam. A lógica de texto fica em `fraus/explicacao.py`; os limites estão em Limitações.

4.8 CONFIANÇA: CONJUNTO CONFORMAL E ABSTENÇÃO

Duas rotas somente leitura, que não gravam e não alteram score, nota nem categoria: `GET /conversas/{id}/confianca` (conjunto de classes e o marcador de abstenção da conversa) e `GET /indicadores/confianca` (agregado, só pelo critério do conjunto; conversas de outra régua ficam fora e vêm contadas em `defasadas`). A lógica pura está em `fraus/conformal.py`; as métricas de calibração, em `fraus/calibracao.py`.

O artefato é `fraus/dados/conformal_fusor.json`, versionado, fora de `modelos/`, chaveado pela régua (`regua_do_fusor`: fusor mais regra de cortesia). Contrato de falha: sem artefato, artefato malformado ou régua diferente da vigente, as duas rotas respondem **503 com o motivo**; no detalhe, uma conversa pontuada por outra régua responde 409. Conjunto vazio conta como abstenção. Conversa sem score tem conjunto `null` e não entra na contagem. Indicadores e demais rotas não dependem do artefato. Limites em Limitações.

4.9 SINALIZADORES POR REGRA

`sinalizadores` é derivado na leitura, em `_com_derivacoes`, por `fraus/sinalizadores.py`: cinco regras de risco sobre a estrutura da conversa, sem gravar coluna nem chave de payload e sem ler o texto ou o relógio da fala humana. Conversa sem fala do cliente devolve `null` (não avaliada), nunca lista vazia. `GET /conversas?sinalizador=<regra>` filtra e só então pagina. Limites em Limitações.

4.10 IMPACTO POR TEMA E DETERIORAÇÃO

`GET /operacao/radar` traz, em cada tema, o impacto no NPS inferido com IC por bootstrap (`fraus/impacto.py`): associação, não causa, nulo abaixo de 30 conversas com sinal em cada lado. `GET /operacao/replay/{id}` traz o início retrospectivo da deterioração (`fraus/deterioracao.py`, campo `indice_virada`) sobre os scores por prefixo que o replay já calcula: derivada na leitura, nada gravado, nunca alerta ao vivo. Limites em Limitações.

4.11 ONDE CADA COISA MORA

CAMINHO	RESPONSABILIDADE
`fraus/modelos.py`	`Conversa` / `Mensagem` — o modelo canônico
`fraus/ingest/`	adaptadores de entrada, um por formato
`fraus/sinais/`	as oito famílias de sinal
`fraus/sinais/curadoria.py`	o que o analista ensinou ao léxico — vence o SentiLex
`fraus/fusor.py`	`NOMES_FEATURES` (39) e o `Fusor`
`fraus/indicadores.py`	NPS, CSAT, contenção, falso containment, série
`fraus/explicacao.py`	frases por template a partir da decomposição do fusor, e os drivers agregados
`fraus/evidencia.py`	evidência fraca — a cabeça tracejada
`fraus/deriva.py`	deriva de distribuição — a invariante 10 em runtime
`fraus/cortesia.py`	"ok, obrigado" sozinho é sem sinal, não promotor
`fraus/api/`	FastAPI: `criar_app` (fábrica) e as rotas por domínio
`fraus/api/contexto.py`	faixas, curadoria e régua vigentes, lidas por requisição
`fraus/api/repontuacao.py`	repontuar em segundo plano, com progresso e 409 se já rodando
`fraus/api/vazao.py`	teto de `/auth/*` por IP e de `/ingestao` por fonte
`dashboard/`	Next.js — ver Design

5. Treinamento dos modelos do Fraus

> **COMO LER ESTA PÁGINA (30/09/2026)**
>
> Ela é o **diário técnico do treino** e preserva a ordem em que as coisas aconteceram, inclusive números que já foram superados. O estado **atual** é: fusor de **39 features** (a ironia saiu do vetor em 04/09/2026 e `incongruencia_situacao_negativa` entrou: 40 − 2 + 1), espera em `log1p` e acurácia de 0,947 no holdout do notebook 02 — ver o cartão do modelo. As menções a 35, 38 e 40 features abaixo são história, não o contrato vigente.

#	NOTEBOOK	O QUE PRODUZ	GPU?	ESTADO
1	`notebooks/01_treino_bertimbau.ipynb`	`modelos/bertimbau-satisfacao/` — classificador de texto (3 classes), fine-tune do `neuralmind/bert-base-portuguese-cased` sobre o B2W-Reviews01	**sim**	pronto
2	`notebooks/02_treino_fusor.ipynb`	`modelos/fusor.joblib` e `modelos/importancias.json` — a regressao logistica que funde os sinais	nao	pronto
3	`notebooks/03_treino_emocao.ipynb`	`modelos/bertimbau-emocao/` e `modelos/metricas_emocao.json` — classificador das sete classes de emocao	**sim**	pronto
4	`notebooks/04_treino_ironia.ipynb`	`modelos/bertimbau-ironia/` e `modelos/metricas_ironia.json` — classificador binario de ironia	**sim**	pronto; o padrão (`ORIGEM = 'idpt_publico'`) baixa as fontes públicas originais, e o pedido aos organizadores do IDPT é alternativa
5	`notebooks/05_treino_multitarefa.ipynb`	candidato ONNX com um encoder e três cabeças	**sim**	candidato, não promovido
6	`notebooks/06_exportar_laya_onnx.ipynb`	ZIP do Laya em ONNX INT8 para o bundle externo da Vercel	nao	carregado quando `FRAUS_IRONIA_BACKEND=laya-onnx` (ver `deploy-vercel.md`)
7	`notebooks/07_treino_laya.ipynb`	`modelos/laya-fraus` — fine-tune do Laya em emoção e ironia, comparado ao BERTimbau	**sim**	**histórico**: executado em 02/10/2026, não promovido; não roda de novo sem rever a receita

O notebook 02 **depende do artefato do 01**: ele carrega o BERTimbau treinado para extrair o sinal de texto de cada conversa. A primeira celula do 02 falha com erro explicito se `modelos/bertimbau-satisfacao` nao estiver no Drive.

Os notebooks 03 e 04 sao **independentes entre si e do 01** — treinam cabecas separadas, em corpora proprios. O 02 e que passa a depender dos tres, porque e ele que monta o vetor de features completo. Ordem de execucao: 01, 03 e 04 em qualquer ordem, e o 02 por ultimo.

O sinal lexico (`fraus/sinais/lexico.py`) **nao tem notebook**: o SentiLex-PT nao e treinado, e recurso pronto, versionado em `fraus/dados/sentilex_pt02.csv`. Ver Sinal lexico no fim.

As tres cabecas BERTimbau e o `fusor.joblib` sao obrigatorios para servir predicao. `fraus/api/main.py` sobe sem carrega-los: o `ProvedorDeMotor` carrega tudo na primeira rota de modelo (ou no aquecimento) e propaga o erro sem fallback — servir predicao sem modelo carregado e pior do que estar fora do ar. `scripts/api_demo.py` usa o motor real quando os pesos estao no disco e so cai no motor duble, anunciado no boot e em `/saude`, quando falta um artefato; ele nao serve para producao.

O treino roda inteiramente no Google Colab, fora deste repositorio — nao ha teste automatizado local para essas etapas.

5.1 COMO ABRIR NO COLAB

1. Va em colab.research.google.com.
2. `Arquivo > Fazer upload de notebook` e selecione o notebook (ou abra direto do GitHub se o repositorio estiver acessivel a partir da sua conta).

5.2 NOTEBOOK 01 — CLASSIFICADOR DE TEXTO

5.2.1 Exigencia de GPU

O notebook so funciona com acelerador de GPU habilitado. Antes de rodar qualquer celula:

`Ambiente de execucao > Alterar tipo de ambiente de execucao > GPU`

A primeira celula do notebook faz `assert torch.cuda.is_available()` e falha com uma mensagem explicita se o ambiente de execucao estiver sem GPU.

5.2.2 Onde o checkpoint fica no Drive

O notebook monta o Google Drive (`drive.mount('/content/drive')`) e usa dois diretorios dentro dele, para sobreviver a uma queda de sessao do Colab — o mesmo padrao ja usado no notebook de RVC do projeto Neuro-ai:

- `/content/drive/MyDrive/fraus/checkpoints` — checkpoints intermediarios salvos pelo `Trainer` a cada epoca (`save_strategy="epoch"`, `save_total_limit=2`). Se a sessao cair, o proximo `Trainer.train()` pode retomar a partir do ultimo checkpoint salvo aqui.
- `/content/drive/MyDrive/fraus/modelos/bertimbau-satisfacao` — destino final do modelo treinado, do tokenizador e do `metricas.json`, escritos na ultima celula do notebook.

5.2.3 Para onde copiar o artefato no repo local

Depois que o notebook terminar (celula de avaliacao e export), baixe a pasta `fraus/modelos/bertimbau-satisfacao` do Google Drive e copie o conteudo para:

```
modelos/bertimbau-satisfacao/
```

na raiz deste repositorio. Os arquivos esperados sao:

- `config.json`
- `model.safetensors`
- `tokenizer.json`
- `tokenizer_config.json`
- `special_tokens_map.json`
- `metricas.json` — `{"acuracia": float, "f1_macro": float, "classes": ["insatisfeito", "neutro", "satisfeito"]}`

`fraus/sinais/texto.py` carrega esse diretorio para servir o classificador — e o notebook 02 tambem, para extrair o sinal de texto das conversas de treino do fusor.

Importante: `modelos/` esta no `.gitignore` deste repositorio. O artefato do modelo treinado (pesos, tokenizer, metricas) **nao vai para o git** — ele fica local, versionado apenas via o checkpoint no Drive.

5.2.4 Mapeamento de rotulo

O rotulo de satisfacao vem do campo `overall_rating` (nota de 1 a 5) do B2W-Reviews01:

`OVERALL_RATING`	ROTULO	INDICE
1–2	insatisfeito	0
3	neutro	1
4–5	satisfeito	2

O corpus e balanceado por subamostragem da classe majoritaria antes do split de treino/teste, porque as notas 4-5 dominam o corpus e enviesariam o classificador.

5.3 NOTEBOOK 02 — FUSOR DOS TRES SINAIS

Roda DEPOIS do 01, sem GPU. O que ele faz, em sequencia:

1. monta o Drive e confere `modelos/bertimbau-satisfacao` (falha alto apontando o notebook 01 se faltar);
2. clona este repositorio e instala o pacote `fraus` — a extracao de features usa o MESMO codigo da API (`fraus.fusor.montar_features`), nunca uma reimplementacao;
3. carrega o B2W-Reviews01 e rotula por `recommend_to_a_friend` (ver abaixo);
4. costura as frases em conversas sinteticas com `fraus.ingest.simulador.gerar_lote`, deterministico por semente, com latencia log-normal e emoji calibrados por rotulo;
5. extrai as features de cada conversa com o BERTimbau do notebook 01 carregado — hoje sao **39** (ver Contrato de features);
6. treina o `Fusor` (`treinar(exemplos, rotulos)`);
7. avalia num conjunto de teste separado — conversas geradas com outra semente e a partir de frases disjuntas — imprimindo acuracia e F1-macro;
8. exporta `fusor.joblib` (via `Fusor.salvar`) e `importancias.json` (o retorno de `Fusor.importancias()`, que vira o grafico "qual sinal pesou mais" da apresentacao).

Os dois arquivos saem em `/content/drive/MyDrive/fraus/modelos/`. Baixe para `modelos/` na raiz do repositorio local, ao lado de `bertimbau-satisfacao/`.

5.3.1 Rotulo do fusor: `recommend_to_a_friend`

Esta e a etapa em que o campo `recommend_to_a_friend` do B2W-Reviews01 entra. A spec (secao 5.1) o declara como a **ancora do NPS inferido** — "sem ele, o NPS da dashboard seria metrica inventada" — e o alvo do fusor e justamente a categoria de NPS que a dashboard exibe. O notebook 01 usou `overall_rating` porque treinava sentimento de texto; o 02 usa a recomendacao porque treina NPS.

CAMPO	VALOR	CLASSE	INDICE
`recommend_to_a_friend`	`Yes`	satisfeito	2
`recommend_to_a_friend`	`No`	insatisfeito	0
`overall_rating`	`3`	neutro	1

Por que a linha do neutro existe. `recommend_to_a_friend` e binario, e as classes do projeto sao tres. Um fusor de duas classes zeraria o termo do neutro no score (`100 * (P(satisfeito) + 0.5 * P(neutro))`) e empurraria todo atendimento morno para uma das pontas, inflando o NPS nos dois sentidos. A alternativa seria treinar em duas classes e recortar o neutro por uma faixa de score — mas isso trocaria um rotulo observado por um limiar arbitrado a mao depois do treino. Aqui o neutro e `overall_rating == 3`, um rotulo do corpus, e a nota 3 tem precedencia sobre a

recomendacao para que a mesma resenha nao caia em duas classes. As duas pontas — as unicas que contam no NPS, ja que neutros nao entram na formula — seguem ancoradas em `recommend_to_a_friend`.

As tres classes sao subamostradas ao tamanho da menor (a nota 3 e minoria no corpus), o mesmo balanceamento do notebook 01.

5.3.2 Limitacao metodologica a declarar no relatorio

O fusor e treinado sobre conversas SINTETICAS: nenhum corpus publico de resenha PT-BR tem timestamps de dialogo, entao a latencia, a escalacao e o abandono saem de distribuicoes calibradas pela literatura de live chat, nao de atendimento observado. O texto e real (B2W), o tempo nao e.

As metricas do notebook 02 continuam OTIMISTAS mesmo depois da correcao descrita abaixo: elas medem que o fusor aprendeu a combinar os sinais no dominio sintetico, nao que ele acertaria essa taxa em atendimento real.

5.3.3 O vazamento do primeiro fusor — 13/08/2026

O primeiro fusor treinado marcou **99,3% de acuracia e F1-macro 0,993**, e era inutil. Vale registrar porque o modo de falha e silencioso e pode voltar.

O sintoma. Carregado no repositorio local e alimentado com conversas escritas a mao, o fusor devolvia praticamente a mesma nota para tudo:

	LATENCIA 5S	30S	200S
texto otimo (BERTimbau: 0,96 satisfeito)	52,48	51,32	50,02
texto pessimo (BERTimbau: 0,98 insatisfeito)	49,94	49,92	49,84

Dois textos que o classificador de texto separa com mais de 95% de confianca saiam com **2,5 pontos** de diferenca, ambos colados em 50 e ambos classificados como neutro.

A causa. O `PERFIL_POR_ROTULO` do simulador amostrava latencia de faixas **disjuntas** — insatisfeito 60-400s, neutro 15-60s, satisfeito 3-15s. Sem sobreposicao nenhuma, a latencia *era* o rotulo: dava para acertar a classe olhando so o relogio, sem ler uma letra do texto. A regressao logistica fez exatamente isso. Os quatro maiores pesos aprendidos eram todos de tempo (`latencia_p90_s` 2,06 · `latencia_mediana_s` 2,03 · `latencia_primeira_resposta_s` 1,91 · `duracao_total_s` 1,52), contra 1,39 e 1,12 das features de texto. Fora da variedade que ele decorou, o modelo respondia "neutro" para tudo.

Os 99,3% nao mediam fusao. Mediam que o simulador vazava o gabarito.

Um segundo achado, do mesmo diagnostico. As features `emoji_score_medio`, `emoji_frac_positivos` e `emoji_frac_negativos` tinham coeficiente **0,0000**: o simulador nunca emitia emoji, entao elas eram constantes no treino e um dos tres sinais da arquitetura nao contribuia nada.

A correcao (`fraus/ingest/simulador.py`, coberta por `tests/test_simulador.py`):

- a latencia passou a ser **log-normal** com caudas que se cruzam. As medianas continuam ordenadas, como manda a literatura de live chat, mas ~11% das conversas satisfeitas sao mais lentas que a mediana das insatisfeitas, e vice-versa. Existe atendimento rapido que termina mal;
- as falas do cliente passaram a levar **emoji do lexicon**, com cruzamento deliberado — cliente irritado manda 🙏 (+0,418), cliente satisfeito manda 😩 (-0,368) — para o emoji nao virar o proximo gabarito.

Como saber se voltou. Acuracia alta e sintoma que pede investigacao, nao veredito. Depois da correcao o notebook 02 marcou **0,96**, e desta vez o numero se sustentou: os pesos aprendidos mudaram de lugar.

FEATURE	FUSOR VAZADO	FUSOR CORRIGIDO
`emoji_score_medio`	**0,0000** (morto)	**1,6662** (1º)
`texto_prob_satisfeito_media`	1,3858	1,6026 (2º)
`texto_prob_insatisfeito_media`	1,1212	1,4722 (3º)
`latencia_p90_s`	**2,0575** (1º)	0,0674
`latencia_mediana_s`	2,0329	0,1621
`latencia_primeira_resposta_s`	1,9068	0,0138

O mesmo teste de sanidade que expos o vazamento — mesmo texto, so mudando o relogio — passou de 2,5 pontos de separacao para **99,6**: texto otimo pontua 99,98 e texto pessimo 0,35, com a latencia mexendo fracoes de ponto. O sinal de tempo virou o modificador fraco que ele deve ser.

O criterio, entao, nao e o numero sozinho. Diante de acuracia alta, olhe os PESOS: se as features que lideram forem as que carregam o conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais (tempo, contagem de turnos), procure o vazamento.

5.3.4 O fusor de 35 features — 24/08/2026

O contrato subiu de 16 para 35 features em 21/08 (emocao, lexico, ironia e estilo entraram no vetor) e o **artefato ficou tres dias para tras**. Nesse intervalo a API real nao subia: o `StandardScaler` de 16 rejeitava o vetor de 35 na carga. Falha alta e explicita, que e o comportamento que as invariantes 7 e 9 pedem — mas ainda assim tres dias de API parada por artefato desatualizado.

O notebook 02 rodou de novo e o artefato daquele momento passou a ter `n_features_in_ = 35`, na ordem exata de `NOMES_FEATURES`. (Historico: o artefato vigente hoje tem 38 — ver as secoes de 03/09 e 04/09 abaixo.)

A acuracia caiu de 0,96 para 0,93, e a queda e o resultado saudavel. O fusor de 16 features media um problema mais facil. Dezenove features novas entraram, tres delas vindas de uma cabeca de ironia com

vazamento de corpus conhecido e nao consertado — um numero MENOR e o esperado. Pela regra desta secao, o que decide nao e o numero e sim onde os pesos foram parar:

FEATURE	FUSOR DE 16	FUSOR DE 35
`texto_prob_satisfeito_media`	1,6026 (2º)	**1,6572 (1º)**
`texto_prob_insatisfeito_media`	1,4722 (3º)	1,3867 (2º)
`emoji_score_medio`	**1,6662 (1º)**	1,3354 (3º)
`emoji_frac_positivos`	0,8430	0,8098
`emocao_surpresa_media`	—	0,6655
`emocao_desprezo_derivado`	—	0,4757
`latencia_mediana_s`	0,1621	0,1200
`latencia_p90_s`	0,0674	0,1419
`duracao_total_s`	0,4964	0,3397

O topo continua sendo **texto e emoji**, e as features de tempo continuam no rodape, onde devem estar. As de emocao entram no meio da tabela com peso real sem deslocar o conteudo — `desprezo_derivado`, que e a diade calculada e nao uma classe do modelo, puxa 0,48. Nao ha sinal de vazamento novo.

A divida que este fusor assume. Ele foi treinado com a cabeca de ironia vazando (ver a pendencia 1 do README: 6 em 10 frases sinceras saem marcadas como ironicas). `ironia_prob_media` e `ironia_prob_max` pesam 0,30 e 0,31 — nao e desprezivel. **Retreinar a ironia obriga a retreinar o fusor de novo**, e essa ordem esta registrada aqui para nao se perder.

Teste de sanidade ponta a ponta, com os tres BERTimbau carregados, sobre conversas do simulador: rotulo insatisfeito pontua ~0, neutro ~73–75, satisfeito ~99, e as tres categorias de NPS saem certas em 9 de 9. **Isso nao e evidencia de qualidade** — sao conversas do proprio gerador sintetico, o dominio em que o fusor foi treinado. O numero honesto continua sendo 0,93 no conjunto de teste separado, e a ressalva do topo desta secao continua valendo: as metricas do notebook 02 medem fusao no dominio sintetico, nao acerto em atendimento real.

5.3.5 Consequencia no NPS: a classe neutra cai em detrator — HISTORICO (fusor de 16 features, ate 24/08/2026)

Esta secao mede o fusor de 16 features, NAO o vigente. Ela ficou aqui sem data enquanto o artefato foi trocado, e passou a ler como se descrevesse o fusor atual — que e o defeito mais caro deste projeto. Fica registrada porque o raciocinio sobre o score continua valendo, e porque a medicao dela e o contraponto do que o fusor de 35 features entrega.

A consequencia DEIXOU de valer com o fusor de 35 features. Base: o teste de sanidade ponta a ponta da secao anterior, com os tres BERTimbau carregados — rotulo neutro pontua **~73–75**, cai na faixa 7–8 e as tres categorias de NPS saem certas em 9 de 9. Ou seja, o neutro nao cai mais em detrator. Ressalva do mesmo

tamanho: aquele teste corre sobre conversas do proprio simulador, o dominio em que o fusor foi treinado, entao ele desmente a medicao abaixo no dominio sintetico e nao promete nada sobre atendimento real.

Medido em 90 conversas do simulador, 30 por classe, o fusor corrigido separa bem — medianas **0,11 / 50,99 / 99,03** por rotulo verdadeiro. Mas a categoria de NPS derivada sai **67% detrator · 29% promotor · 4% neutro**, com **NPS -38** num lote equilibrado por construcao.

Nao e falha do treino. O score e `100 * (P(satisfeito) + 0.5 * P(neutro))`, entao conversa classificada com certeza como neutra pontua 50, vira nota 5 e cai na faixa 0-6. A faixa neutra do NPS (7-8) so seria alcancada com `P(satisfeito)` entre 0,4 e 0,8 — um empate entre classes, nao uma neutralidade confiante. As tres classes do modelo e as tres categorias do NPS foram decididas em lugares separados e nunca se encontraram.

Decisao de projeto: manter e declarar. A alternativa avaliada era subir o peso do neutro de 0,5 para 0,75 (o centro da faixa passiva do NPS), o que faria neutro puro pontuar 75, virar nota 8 e cair em neutro — alinhando as tres classes as tres categorias. Ficou registrada aqui e na docstring de `Fusor.pontuar` para nao ser "corrigida" por engano.

5.3.6 Retreino do fusor apos as features de incongruencia (03/09/2026)

O contrato subiu de 35 para 40 features: a familia `incongruencia_*` (`incongruencia_polaridade`, `incongruencia_emoji_texto`, `incongruencia_marcador_contraste`, `incongruencia_hiperbole`, `incongruencia_aspas_ironicas`) entrou em `NOMES_FEATURES`, anexada ao FIM da lista para preservar a ordem das 35 antigas. O fusor salvo em `modelos/` foi treinado com 35 e `vetorizar` agora produz 40 — ele NAO serve mais. `Fusor.carregar` (`joblib.load` puro, sem validacao de forma) carrega esse artefato sem erro nenhum -- a API real SOBE com ele. O erro de dimensao no `StandardScaler` so aparece na primeira pontuacao de verdade (`/ingestao` ou `/conversas/importar`), como HTTP 500, nao na carga do servidor. Isso e risco de demonstracao ao vivo, nao uma trava de subida -- ver a tabela do Fusor no README para o detalhe medido. A intencao da invariante 7 (falha alta e explicita) so se cumpre na metade "explicita"; a metade "na carga" fica para o retreino resolver o descompasso, nao para uma validacao que este codigo ainda nao tem.

O corpus sintetico tambem mudou, e isso importa mais do que a contagem de features. `FRASES_POR_ROTULO`, em `fraus/ingest/simulador.py`, ganhou frases novas em duas levas para curar dois vazamentos de rotulo — o mesmo tipo de falha silenciosa que ja custou o primeiro fusor (ver acima), so que desta vez em features que ainda nao estavam no vetor:

- **Intensificadores.** Antes, `incongruencia_hiperbole` era nao-zero na maior parte das conversas do rotulo satisfeito e ZERO nos rotulos insatisfeito e neutro — um previsor unilateral quase perfeito de "satisfeito". A literatura inclui hiperbole na deteccao de ironia justamente porque elogio hiperbolico e a forma classica da ironia, entao o fusor teria aprendido a feature ao contrario do que ela significa. Depois da correcao, medido com `gerar_lote(FRASES_POR_ROTULO, 180, semente=7)` — o mesmo lote e semente da guarda `test_nenhuma_feature_e_previsor_unilateral` em `tests/test_simulador.py`: `insatisfeito 34/60 | neutro 0/60 | satisfeito 39/60`.
- **Negacao.** `lexico_frac_negados` era nao-zero so no rotulo insatisfeito e ZERO nos outros dois — previsor unilateral de "insatisfeito", pre-existente desde antes da familia `incongruencia_*` e nunca notado por falta de guarda na familia `lexico_*`. Depois, medido com o mesmo lote e semente acima: `insatisfeito 15/60 | neutro 17/60 | satisfeito 21/60`.

Consequencia pratica: o notebook 02 precisa REGERAR o corpus sintetico, nao reusar um lote cacheado de uma execucao anterior. O notebook 02 ja gera o lote do zero a cada execucao — `gerar_lote` roda

na propria celula, com semente fixa, e nao le nenhum artefato salvo de rodada passada — entao rodar o notebook de novo, sem alterar nada, ja traz o corpus corrigido. O risco e so para quem tiver guardado um `.csv`/`.pkl` de conversas de uma sessao antiga do Colab e tentar reaproveitar: um lote assim traz os dois vazamentos de volta.

O notebook ja importa o contrato do pacote, nao repete a lista. A celula marcada com o comentario `# 5. Os SETE sinais de cada conversa, pelo MESMO codigo que a API usa` (indice 8 do `.ipynb`, contando a partir de 0) faz `from fraus.fusor import NOMES_FEATURES, montar_features` e chama `montar_features` com os tres classificadores — as cinco features novas entram sozinhas, sem editar o notebook. Isso ja era assim antes desta rodada (o vazamento do ramo obsoleto registrado na celula 2 foi outra causa, a de importar de um ramo desatualizado do repositorio, nao de reimplementar a lista) — nao ha lista duplicada para divergir aqui.

Passos, no Colab:

1. Conferir que a celula do clone do repositorio (`RAMO`) aponta para `main` com a Task 5 mergeada, e rodar o notebook do inicio — sem tentar reaproveitar nenhum lote de conversas salvo de uma execucao anterior.
2. Antes de subir ao Colab (ou depois, localmente), rodar as guardas de vazamento: `uv run pytest tests/test_simulador.py -k unilateral`. Elas cobrem as familias `lexico_*`, `emoji_*`, `estilo_*` e `incongruencia_*` contra previsor unilateral e precisam estar verdes antes do treino valer a pena.
3. Conferir a acuracia contra a do fusor de 35 features (0,93). **Acuracia alta demais e sintoma, nao vitoria** — se saltar muito acima do valor anterior, suspeite de vazamento antes de comemorar, releia a secao "O vazamento do primeiro fusor" acima e confira os PESOS, nao so o numero: se as features que lideram forem as de conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais, procure o vazamento.
4. Conferir os coeficientes das cinco features novas em `Fusor.importancias()` / `Fusor.eixo_global()`.

CORRECAO DE 04/09/2026, e a licao vale mais que o numero: este passo dizia para esperar peso proximo de zero em `incongruencia_marcador_contraste` e `incongruencia_aspas_ironicas`, porque as duas sao constantes em zero no corpus do SIMULADOR. A previsao estava errada, e o erro foi confundir dois corpora diferentes. O simulador fornece a ESTRUTURA temporal (latencia, turnos, escalacao) a partir de seis frases por rotulo; o TEXTO do treino e o B2W-Reviews01, resenha real de e-commerce, onde marcador de contraste e aspas ironicas ocorrem normalmente. Medir uma feature de texto no gerador de estrutura e olhar para o corpus errado.

Os pesos reais do primeiro retreino de 40 features, no eixo satisfeito-menos-insatisfeito:

FEATURE	PESO
`incongruencia_emoji_texto`	−0,95
`incongruencia_hiperbole`	−0,65
`incongruencia_marcador_contraste`	−0,38
`incongruencia_polaridade`	−0,37
`incongruencia_aspas_ironicas`	−0,15

As cinco sairam NEGATIVAS, que e a direcao que a literatura preve: mais incongruencia empurra a nota para insatisfeito. A hiperbole em −0,65 e a evidencia de que o cruzamento do corpus (secao acima) funcionou — sem ele ela teria aprendido o sinal contrario.

O que continua valendo da versao anterior deste passo: as guardas de vazamento medem o SIMULADOR e sao sobre a estrutura, nao sobre o texto do B2W. `incongruencia_hiperbole` melhorou mas nao chegou aos tres: depois da correcao ela dispara em DOIS dos tres rotulos — `insatisfeito 34/60` e `satisfeito 39/60` (numeros medidos acima, na secao do retreino) —, e `neutro` continua em `0/60`. Isso e limitacao declarada, nao vazamento remanescente: fala neutra raramente intensifica termo polar com hiperbole, e a decisao foi nao forcar uma frase artificial no gerador so para a classe deixar de ficar muda. A guarda `test_nenhuma_feature_e_previsor_unilateral` (`tests/test_simulador.py`) tolera isso por construcao — ela so falha quando a feature dispara em EXATAMENTE UM rotulo, entao 2-de-3 passa verde. Isso e posicao aceita sobre o corpus sintetico atual, nao descuido: fica registrado aqui para o proximo leitor nao achar que a guarda promete "sinal nos tres rotulos" quando ela so promete "nao previsor unilateral de UM rotulo so". A guarda nao foi alterada. 5. Copiar `fusor.joblib` e `importancias.json` para `modelos/` na raiz do repositorio local (guardando o artefato anterior como `.bak-<N>`, onde N e o numero de features dele) e entao, nesta ordem: `uv run python scripts/conferir_fusor.py` e `uv run pytest -q` inteiro. O laudo faz automaticamente os passos 3 e 4 desta lista — ver "Conferencia do artefato" abaixo.

5.3.7 A ironia sai do vetor (04/09/2026)

Medicao sobre o proprio corpus de treino do fusor (B2W-Reviews01, amostra equilibrada de 500 resenhas por rotulo, semente 20260904), rodando so a cabeca de ironia:

```
resenha NEGATIVA (nao): media P(ironia) = 0,1012  mediana 0,0013  frac(P>0,5) = 0,092
resenha POSITIVA (sim): media P(ironia) = 0,7146  mediana 0,9687  frac(P>0,5) = 0,740
                                        diferenca (sim-nao) = +0,6134
```

A cabeca, treinada no IDPT 2021 (tweet e noticia), aplicada a resenha de e-commerce funciona na pratica como **detector de sentimento positivo**: marca 74% das resenhas satisfeitas como ironicas contra 9% das insatisfeitas. O fusor de 40 features aprendeu **+0,77** de peso para `ironia_prob_media` no eixo satisfeito-menos-insatisfeito — mais ironia empurrando para SATISFEITO, o inverso do que o nome da feature promete. O fusor de 35 ja tinha o mesmo sinal (+0,48), entao nao e regressao desta rodada: e o mesmo vazamento medido de novo, agora com numero.

Consequencia observavel: a frase canonica de ironia deste projeto — "que atendimento maravilhoso, so esperei 3 horas" — pontuava **99,98 / nota 10 / promotor** no fusor de 40, porque as duas features de ironia empurravam para cima em vez de para baixo. (A cabeca isolada acerta essa frase: 0,998 nela, 0,002 num elogio sincero. Quem lia ao contrario era o fusor, e so por causa do corpus.)

Decisao: `ironia_prob_media` e `ironia_prob_max` saem de `NOMES_FEATURES`. Feature que mede outra coisa que nao o nome dela e pior que feature ausente — ela nao adiciona sinal, duplica `texto_prob_satisfeito_*` com ruido, e inverte o caso que o projeto usa como exemplo de manual. A cabeca de ironia CONTINUA carregada e obrigatoria (invariante 7) e continua sendo lida por mensagem e exibida na dashboard, onde e honesta e ja vem com a ressalva de confiabilidade que a tela mostra — o que muda e que ela para de pontuar. O retreino com 38 features foi executado no mesmo dia; ver a secao seguinte.

Nota sobre a janela entre a mudanca do contrato e o artefato novo. Enquanto `modelos/fusor.joblib` era o artefato de 40 e o contrato ja estava em 38, a API **nao subia** — `Fusor.carregar` passou a comparar `n_features_in_` com `len(NOMES_FEATURES)` e levantar `FusorIncompativelError` (PR #29, 04/09/2026). Antes dessa validacao, o mesmo desencontro deixava a API subir normalmente e estourar HTTP 500 so na primeira pontuacao real — falha tardia, no pior momento possivel. A validacao troca isso por falha alta e explicita na subida, que e o que a invariante 7 pede.

5.3.8 O fusor de 38 features — 04/09/2026

Notebook 02 rodado de novo com a ironia fora do vetor. **Acuracia 0,9433** (f1_macro 0,9436), contra 0,9467 do fusor de 40. A queda de ~0,003 e o resultado saudavel e esperado: as duas features removidas *estavam* ajudando o classificador — so que como detectores de sentimento positivo duplicando `texto_prob_satisfeito_*`, nao como medida de ironia. Perder um pouco de acuracia trocando uma feature que mente por nenhuma feature e o negocio certo.

Conferido com `uv run python scripts/conferir_fusor.py` (ver secao "Conferencia do artefato" abaixo): contrato 38/38, `classes_ = [0, 1, 2]` (invariante 8), nenhuma feature com peso perto de zero, e as cinco de incongruencia mantiveram a direcao negativa que a literatura preve:

FEATURE	FUSOR DE 40	FUSOR DE 38
`incongruencia_emoji_texto`	−0,95	−1,03
`incongruencia_hiperbole`	−0,65	−0,64
`incongruencia_polaridade`	−0,37	−0,37
`incongruencia_marcador_contraste`	−0,38	−0,27
`incongruencia_aspas_ironicas`	−0,15	−0,19

Tres pesos com sinal aparentemente invertido, e por que NAO sao vazamento. O laudo sinaliza `texto_prob_satisfeito_ultima` (−0,50), `emoji_frac_positivos` (−0,18) e `emoji_frac_negativos` (+0,60) como suspeitos, porque o nome promete o sinal contrario. Conferidos contra os artefatos de 40 e de 35, os tres ja estavam assim nos dois — nao e efeito da remocao da ironia. A explicacao e **colinearidade**: `texto_prob_satisfeito_media` pesa +2,81 e `emoji_score_medio` +1,56, e features fortemente correlacionadas com elas recebem coeficiente negativo de correcao na regressao logistica. E o comportamento normal do modelo com features redundantes, nao inversao de semantica como a de `ironia_prob_media`. A diferenca entre os dois casos e verificavel: a ironia foi medida diretamente no corpus e provou medir outra coisa; estas tres continuam medindo o que o nome diz, so nao carregam informacao independente das dominantes. Fica registrado aqui porque o laudo vai sinalizar as tres em todo retreino futuro, e o proximo leitor precisa saber que ja foram investigadas.

O que este retreino NAO resolveu. A frase canonica do projeto continua pontuando alto:

```
"que atendimento maravilhoso, so esperei 3 horas"   score 99,95  nota 10
"otimo, mais uma vez ninguem resolveu nada"         score 99,74  nota 10
"resolveu rapido, muito obrigado! adorei"           score 97,82  nota 10
```

As cinco features de incongruencia dao **0,0 nas duas frases ironicas** e disparam na satisfeita de verdade (`polaridade` 0,5, `hiperbole` 0,5) — ou seja, hoje elas pegam entusiasmo genuino e nao pegam ironia. Nao e regressao: e o limite estrutural das cinco, todas funcao de `anotar_texto`/emoji, e "so esperei 3 horas" nao tem nenhuma palavra polar no SentiLex — e negativo por pragmatica. O diagnostico completo e a rota proposta (`incongruencia_situacao_negativa`, que levaria o vetor a 39 e exige mais um retreino) estao em `docs/superpowers/specs/2026-09-04-incongruencia-implicita-design.md`.

Corrigindo a previsao registrada na secao anterior: esperava-se que a frase deixasse de ser empurrada para cima com a saida da ironia. Nao foi o que aconteceu — o `texto_prob_satisfeito_media` da propria cabeca de texto e 0,858 nela, e domina o eixo sozinho. A remocao consertou o vetor; nao tocou nesta frase.

5.3.9 O contrato sobe para 39: incongruencia implicita (04/09/2026)

`incongruencia_situacao_negativa` entrou em `NOMES_FEATURES` -- elogio convivendo com situacao negativa de atendimento na mesma fala. E a unica das seis que alcanca a frase canonica do projeto, porque nao depende de um segundo termo polar no lexicon.

O retreino do notebook 02 e OBRIGATORIO antes de a API voltar a subir. O artefato de 38 nao carrega mais: `Fusor.carregar` valida a dimensao e levanta `FusorIncompativelError`. Isso e o comportamento desejado, nao um bug.

Medido ANTES de aceitar, no B2W-Reviews01 (2000 resenhas por rotulo, semente 20260904) -- que e o corpus que da o TEXTO do treino:

```
insatisfeito  82/2000 = 4,10%
neutro        53/2000 = 2,65%
satisfeito    35/2000 = 1,75%
```

Dispara nos tres rotulos, gradiente suave, razao 2,3:1 entre os extremos. Compare com a cabeca de ironia que saiu do vetor no mesmo dia (74% contra 9%, razao 8:1): aquilo era disjuncao pratica, isto e sinal.

Uma armadilha de metodo que quase repeti. Medir esta feature no corpus do SIMULADOR dava zero nos tres rotulos, e eu quase concluí que ela nasceria com peso zero. Errado, e pelo mesmo motivo registrado na correcao de 04/09 mais acima: `FRASES_POR_ROTULO` **nao treina o fusor**. O notebook 02 usa 400 frases do B2W por classe e pega do simulador so a ESTRUTURA da conversa. O simulador importa para a guarda `test_nenhuma_feature_e_previsor_unilateral`, nao para o peso aprendido. As frases dos tres rotulos foram cruzadas mesmo assim -- sem isso a feature ficava constante em zero la e a guarda passaria por vacuidade, que e o modo de falha silencioso dela.

5.3.10 Conferencia do artefato

```
uv run python scripts/conferir_fusor.py             # laudo completo
uv run python scripts/conferir_fusor.py --sem-sonda  # pula as frases-sonda
```

Laudo do artefato vigente em `modelos/`, em seis itens: contrato, ordem das classes, sinal dos pesos por familia, pesos perto de zero, tres frases-sonda e as metricas do treino. Instrumento, nao botao — nao escreve nada, no modelo de `scripts/medir_faixas.py`.

Ele existe porque **acuracia nao denuncia** os dois defeitos que esta semana custaram dois retreinos: o peso invertido de `ironia_prob_media` e o vazamento de `incongruencia_hiperbole`. Os dois so apareceram porque alguem olhou o sinal dos coeficientes e a distribuicao por rotulo — exatamente o que uma pessoa cansada depois de um treino longo pula. Por isso a acuracia nunca e impressa sozinha no laudo: vem sempre com o lembrete da invariante 10, e com alerta explicito acima de 0,97.

5.4 CONTRATO DE FEATURES

`NOMES_FEATURES`, em `fraus/fusor.py`, e a lista canonica. `vetorizar` levanta `KeyError` se faltar chave — nunca zero silencioso (invariante 9).

Hoje sao **39 features**, de SETE familias no vetor (OITO sinais existem no sistema — a ironia continua existindo e sendo lida por mensagem, so nao entra mais aqui, ver a secao acima). O contrato subiu de 16 para 35 em 21/08/2026, quando os notebooks 03 e 04 passaram a existir e a condicao que justificava a espera acabou; de 35 para 40 em 03/09/2026, quando a familia `incongruencia *` entrou (ver Retreino do fusor apos as features de incongruencia acima); e caiu de 40 para 38 em 04/09/2026, quando `ironia_prob_media` e `ironia_prob_max` sairam (ver A ironia sai do vetor acima).

SINAL	MODULO	FEATURES	NO VETOR?
texto	`fraus/sinais/texto.py`	4	sim
emoji	`fraus/sinais/emoji.py`	5	sim
tempo	`fraus/sinais/tempo.py`	7	sim
emocao	`fraus/sinais/emocao.py`	8	sim
lexico	`fraus/sinais/lexico.py`	3	sim
ironia	`fraus/sinais/ironia.py`	2	**nao** (lida por mensagem, nao pontua)
estilo	`fraus/sinais/estilo.py`	6	sim
incongruencia	`fraus/sinais/incongruencia.py`	5	sim

Consequencia: `montar_features` exige DOIS classificadores (texto e emocao); `Motor` continua exigindo os TRES (a ironia entra na atribuicao por mensagem, nao no vetor), e a API nao sobe sem os tres artefatos em `modelos/`. E o comportamento correto da invariante 7.

5.5 NOTEBOOK 03 — CLASSIFICADOR DE EMOCAO

Requisito de banca: distinguir as **sete emocoes humanas**. A cabeca treinada tem sete classes, e a setima emocao e derivada — ver abaixo.

A ordem das classes **nao e redigitada no notebook**: ele importa `NOMES_EMOCOES` de `fraus.sinais.emocao`, a mesma lista que o sinal usa para ler as probabilidades. Redigitar seria o jeito mais facil de treinar `raiva` no indice 2 e le-la no indice 3.

5.5.1 Ordem canonica das classes

`fraus/sinais/emocao.py` fixa a ordem, e ela vale no notebook, no sinal e no fusor. Inverter nao gera erro: atribui a emocao errada em silencio, a mesma armadilha da invariante 8.

INDICE	CLASSE
0	alegria
1	tristeza
2	raiva
3	medo
4	nojo
5	surpresa
6	neutro

5.5.2 Corpus: `go_emotions_ptbr`

Traducao PT-BR do GoEmotions (Demszky et al., 2020 — 58k comentarios do Reddit, 27 emocoes + neutro), com 43.410 exemplos de treino, 5.426 de validacao e 5.427 de teste. Licenca Apache 2.0. As 27 categorias sao reduzidas as seis de Ekman pelo **mapeamento oficial que o proprio GoEmotions publica** — nao por agrupamento inventado aqui.

O arquivo vem no formato CRU do GoEmotions — **uma linha por anotador**, nao por exemplo. Um rotulo conta quando **pelo menos 2 anotadores** o marcaram (a maioria dos exemplos tem 3 anotadores, entao 2 e maioria simples), e exemplos que caem em mais de uma classe de Ekman sao descartados: a tarefa e multiclasse de rotulo unico, e forcar vencedor entre duas emocoes empatadas inventaria rotulo.

O limiar mais alto rende MAIS exemplos, o que so parece contra-intuitivo ate ver o motivo — com 1 anotador quase todo exemplo recebe varios rotulos Ekman e cai fora do filtro de rotulo unico:

LIMIAR	EXEMPLOS COM ROTULO UNICO
1 anotador	17.367
2 anotadores	**46.014**

5.5.3 O desequilibrio, e por que NAO subamostrar

Distribuicao medida com limiar 2:

CLASSE	EXEMPLOS
alegria	19.284
neutro	13.603
raiva	5.155
surpresa	4.339
tristeza	2.601
medo	588
nojo	**444**

Sao **43:1** entre a maior e a menor. Os notebooks 01 e 02 balanceiam por subamostragem, mas aqui isso seria destrutivo: cortar tudo ao tamanho de `nojo` deixaria **3.108 exemplos** para sete classes, jogando fora 93% do corpus.

Escolha: pesos de classe na perda, pela formula `n / (k * n_c)` — a mesma do `class_weight='balanced'` do scikit-learn. Os 46 mil exemplos ficam. Por isso a metrica de selecao do melhor checkpoint e o **F1-macro, nunca a acuracia**: com 43:1, um modelo que chutasse `alegria` sempre marcaria 42% sem ter aprendido nada.

5.5.4 O teto do desprezo — a declarar no relatorio

`nojo` e a classe mais rara do corpus, e o **desprezo e derivado de raiva x nojo**. A qualidade do desprezo fica limitada de cima pela qualidade da pior classe do modelo: se o F1 de `nojo` sair baixo, o desprezo herda isso. O notebook imprime esse teto explicitamente e grava o F1 por classe em `metricas_emocao.json`, para o numero aparecer no relatorio em vez de ser descoberto pela banca.

Limitacao a declarar: a traducao foi feita por maquina (Google Tradutor), sem revisao humana. Por isso a avaliacao reporta tambem o **XED-pt** (7.220 linhas, CC-BY, Plutchik 8 + neutro) como conjunto de teste independente, que nao passou por traducao automatica. O mapeamento Plutchik -> Ekman descarta *anticipation* e *trust*; o resto casa 1:1.

Duas ressalvas sobre o XED-pt: ele e **portugues europeu** ("pa", "artola"), enquanto o treino e PT-BR — diferenca de variedade, nao so de dominio; e o numero dele vai sair **mais baixo** que o interno, o que e esperado e e justamente o ponto, porque e ele que responde "mas nao e so traducao automatica?".

5.5.5 Por que o desprezo NAO e treinado

A setima emocao e o **desprezo**, e nenhum corpus rotulado em portugues o anota. Havia um corpus multilingue sintetico com `contempt` entre os rotulos, e ele foi **recusado de proposito**: treinar seis classes com dado humano e so o desprezo com dado de outra procedencia ensinaria o modelo a reconhecer o *estilo do texto sintetico* em vez do desprezo. E o mesmo vazamento da latencia disjunta, com outra roupa.

O desprezo e derivado da **diade primaria raiva + nojo**, que e como Plutchik (1980) o define — referencia citavel, nao rotulo inventado. A composicao usa media **geometrica**, nao aritmetica: desprezo exige as duas emocoes juntas, e a aritmetica devolveria 0,5 para raiva pura sem nojo nenhum, o que e raiva e nao desprezo.

5.6 NOTEBOOK 04 — CLASSIFICADOR DE IRONIA

Classificacao binaria: 0 nao-ironico, 1 ironico.

> **19/10/2026: a cabeça vigente é a do MultiPICo-PT.** O notebook 04 passou a treinar no MultiPICo (CC BY 4.0, pares post-resposta com irônico e não irônico vindos das mesmas fontes) com a receita nova (`fraus.receita_treino`). Pareado contra a cabeça do IDPT (`docs/notas/2026-10-19-benchmark-ironia.json`): MultiPICo F1 0,511 → 0,660, diferença +0,149 com IC 95% [+0,088; +0,210] e McNemar p = 0,0002; régua de 20 frases 0,394 → 0,520 (falso positivo 7/10 → 2/10; n pequeno, IC cruza zero). O baseline só de estilo dá 0,499 no teste: o corpus não entrega o rótulo pela forma. ECE 0,34 → 0,06 com a temperatura. **O ONNX dela é fp32**: em int8 ela troca de classe em 2 das sondas de `encolher_modelos.py` (`--precisao fp32`, Δp 0,0000). A ironia continua fora do vetor do fusor, então a troca não pede fusor novo. O texto abaixo é o histórico do IDPT.

Corpus: IDPT 2021, a tarefa de *Irony Detection in Portuguese* do IberLEF — 15,2k tweets e 18,4k noticias anotados.

> **08/10/2026: o treino publico do IDPT entrega o rotulo pela fonte.** Os tweets nao-ironicos sao 100% `#economia` de portais de noticia; as noticias sao rotuladas pelo site. Retreinado com a divisao certa, o BERTimbau deu F1 0,994 no teste e P(ironico) = 1,0 em "ok, obrigado" -- e nao foi exportado. Analise, plano e o treino do Laya so em ironia: Ironia: corpus e Laya.

5.6.1 Fontes publicas do treino e teste oficial fechado

O **teste oficial**, anotado para a competicao, continua protegido e deve ser solicitado aos organizadores em https://sites.google.com/inf.ufpel.edu.br/idpt2021/. Ele nunca entra no treino. As fontes publicas usadas para formar o treino do IDPT, porem, estao abertas: os tweets em https://github.com/fabio-ricardo/deteccao-ironia e as noticias em https://github.com/schuberty/PLNCrawler. O notebook baixa essas fontes no runtime por commits imutaveis e valida cada Git blob antes de usar.

O notebook aceita **tres origens** para o corpus, escolhidas na variavel `ORIGEM` da celula 3. Todas alimentam a mesma pasta e a mesma deteccao de esquema, entao trocar de fonte nao muda o resto do notebook:

`ORIGEM`	QUANDO USAR
`idpt_publico` (**padrao**)	baixa as fontes publicas originais, remove hashtags que vazam o rotulo, duplicatas e conflitos
`sintetico`	gera o corpus com `fraus.ingest.gerador_ironia` — fica como ablacao, nao como fonte principal
`drive`	os arquivos ja estao em `DIR_CORPUS` — foi assim que o IDPT entrou, se liberado
`kaggle`	baixa da conta Kaggle; exige `KAGGLE_USERNAME`/`KAGGLE_KEY` nos Secrets do Colab
`upload`	seletor de arquivos do navegador, para corpus que chegou por e-mail

5.6.2 A divisao por autor — conserto de 02/10/2026

A regra da divisao mora em `fraus/divisao.py` e e a mesma nos notebooks 04 e 07: um grupo por autor, e **cada linha sem autor e um grupo proprio**.

Ate 02/10/2026 a regra estava escrita nos notebooks com `autores.astype(str)`, que transforma o campo vazio no autor "nan". As 18.373 noticias nao tem autor: viraram uma pessoa so e cairam inteiras no teste.

	ANTES	DEPOIS
Treino	12.271 linhas, 86% ironicas, so tweets	28.365 linhas, 59% ironicas
Teste	20.872 linhas (63% do corpus), 43% ironicas	4.778 linhas (14%), 59% ironicas

Os numeros de "depois" foram medidos aplicando a regra nova ao corpus real, fora do Colab; os do treino saem quando o notebook rodar.

Consequencias:

- A cabeca de ironia treinada antes desta data **nunca viu uma noticia** no treino e foi avaliada num teste que era 88% noticias. O F1-macro de 0,511 que o notebook 07 recalculou para ela nesse teste mede mudanca de dominio, nao a tarefa.
- `conferir_divisao` agora recusa, com erro, divisao cujo teste foge do tamanho pedido ou cuja proporcao de classe difere entre os dois lados.
- O notebook 04 grava em `metricas_ironia.json` a impressao (SHA-256) dos textos do teste. O notebook 07 para antes do treino se o teste dele nao tiver a mesma impressao: sem isso, frases do treino do BERTimbau cairiam no teste da comparacao.
- Nas noticias a fonte entrega o rotulo (Estadao e sempre nao-ironico; os dois sites de satira, sempre ironicos). Com as noticias dos dois lados, o numero agregado tende a subir por reconhecer o jornal. Os dois notebooks imprimem o resultado **separado por meio**, e o que se le como desempenho em ironia e o dos tweets e o da regua de dominio. Numero alto no agregado e sintoma (invariante 10), nao vitoria.

5.6.3 Candidato Laya

O Laya multilíngue pode substituir apenas a cabeça de ironia, sem alterar satisfação, emoção ou o fusor. Ele é opt-in porque baixa aproximadamente 647 MB de pesos. O backend `laya` exige torch; o `laya-onnx` (30/09/2026) roda a mesma cabeça pelo ONNX Runtime, **sem PyTorch**, e é o que a API serverless usa quando o ZIP da Laya está configurado (ver `docs/deploy-vercel.md`, interno):

```
uv sync --extra laya
FRAUS_IRONIA_BACKEND=laya uv run uvicorn fraus.api.main:app
```

O checkpoint fica preso à revisão declarada em `FRAUS_LAYA_REVISAO`. Antes de promovê-lo, execute a mesma régua de transferência de domínio:

```
uv run python scripts/validar_candidato_ironia.py --backend laya
```

A aba **Modelo → Ironia · Laya** chama somente essa cabeça e não persiste nem altera notas. O tipo de pergunta usado é `choice` com chaves neutras, não `noul`, por causa do viés de rótulo documentado pelos próprios autores.

FINE-TUNING DO LAYA — NOTEBOOK 07 (EXECUTADO EM 02/10/2026, NÃO PROMOVIDO — VER `NOTAS/2026-10-02-TREINO-LAYA.MD`)

O candidato acima responde a frio. `notebooks/07_treino_laya.ipynb` treina o mesmo checkpoint nas duas perguntas do Fraus — emoção (`PERGUNTA_EMOCAO`, em `fraus/treino_laya.py`) e ironia (`PERGUNTA_IRONIA`) — com o script oficial de GPU única do Laya, e o compara com os BERTimbau dos notebooks 03 e 04 nos mesmos exemplos de teste: McNemar exato e IC por bootstrap da diferença de F1-macro (`fraus/comparacao_modelos.py`), mais o XED-pt e a régua de domínio.

Ele não promete 100% de acerto, e número perto disso seria sintoma (invariante 10). No fim, poda o vocabulário de 256 mil tokens para o que o português usa, exporta para ONNX e só adota INT8 se no máximo 1% das decisões mudar.

A última célula grava `comparacao_modelos.json` (acurácia, F1-macro, matriz de confusão, VP/FP/FN/VN por classe, McNemar, IC e latência em CPU) e o mesmo laudo em Markdown. Copiado para `modelos/`, ele aparece na aba **Modelo → Comparação** da dashboard (`GET /modelo/comparacao`), que também lê uma fala com as cabeças carregadas no servidor, lado a lado. Cada leitura do Laya diz se o artefato do servidor é o treinado pelo notebook 07 ou o checkpoint base ("Laya sem treino"): em produção é o base, que lê ironia e, a frio, emoção.

Para o laudo chegar à produção ele é versionado com o código, em `fraus/dados/comparacao_modelos.json`: `modelos/` fica fora do git e, na Vercel, é trocada inteira pelo ZIP dos pesos. A rota serve o laudo de `modelos/` quando ele existe e, na falta, o versionado. O campo opcional `ressalvas` (lista de textos) registra o que se soube sobre a medição depois de feita; ele aparece no topo da aba e do relatório. Os números do laudo nunca são editados à mão — laudo novo é notebook rodado de novo.

Rode primeiro com `MODO_FUMACA = True`. O que foi pesquisado, o que foi medido localmente e o que ainda falta está na nota interna `docs/notas/2026-10-02-treino-laya.md`.

5.6.4 Alternativas ao IDPT — levantamento de 14/08/2026

Registrado para ninguem repetir a busca. O que **nao serve**, e por que:

FONTE	VEREDITO
`arbml/multilingual_irony` (HF)	so `tweet_id` + `label`, **sem texto**; reidratar exige API paga do Twitter
`ramondomiingos/ptbr-irony-idioms-regionalism` (HF)	< 1.000 linhas, e e *benchmark de LLM* com gabarito em prosa — nao e corpus de classificacao
`rafaelanchieta/PiLN` (GitHub)	sao os **modelos** treinados do IDPT, nao o corpus; embeddings dependem de servidor antigo do NILC
IroSvA / SemEval-2018 traduzidos	**desaconselhado**: traducao automatica reintroduz o vazamento de procedencia que ja custou o primeiro fusor

O candidato mais citado e o corpus de **Goncalves et al., BraSNAM 2015** ("Bazinga! Caracterizando e Detectando Sarcasmo e Ironia no Twitter", UFMG), que circula no Kaggle. Lendo o artigo: sao ~2.628 tweets por classe coletados pelas hashtags `#sarcasm` **e** `#irony` e caracterizados com LIWC — ou seja, e um trabalho brasileiro sobre o Twitter **anglofono**. Corpus em ingles. Usa-lo aqui seria o vazamento de procedencia com selo academico.

Verificado tambem e descartado: `OpenNeuro ds004533` ("Ironia VEV") e **neuroimagem** — ressonancia de 40 sujeitos ouvindo frases ironicas, sem nenhuma frase rotulada para treino.

5.6.5 Onde existe corpus PT-BR de ironia (e como pedir)

Dois, os dois **sem download publico**:

1. **Vieira e Silva, A. (2025)** — *Deteccao automatica de ironia por meio de representacoes contextuais*, tese de doutorado em Linguistica, USP, orientacao de Marcos Lopes. Produziu um corpus de ironia em portugues com conversas da rede X: **1.186 exemplos** finais (de 1.200), anotados por **tres voluntarios humanos**, com o contexto da conversa. Achado relevante para o relatorio: os anotadores precisaram do **contexto de producao em ~30% dos casos** — evidencia direta de que ironia em mensagem isolada tem teto baixo, e ela vale citada mesmo sem o corpus em maos.
2. **Projeto IDPT / UFPel** — https://institucional.ufpel.edu.br/projetos/id/u3345, coordenacao de **Larissa Astrogildo de Freitas**. E o grupo por tras do IDPT 2021. Enderece o pedido a coordenacao.

Tamanho importa aqui: 1.186 exemplos anotados a mao nao treinam um BERT do zero, mas sao excelentes como **conjunto de teste independente** — exatamente o papel que o XED-pt cumpre no notebook 03. E a arquitetura de honestidade que o projeto ja usa: treinar no que ha em volume, medir no que ha em qualidade.

5.6.6 O padrao: corpus sintetico

Enquanto nenhum dos dois chega, o notebook gera o proprio corpus com `fraus.ingest.gerador_ironia`. E o mesmo impasse do sinal de tempo e a mesma saida ja aceita: o fusor treina em conversas sinteticas porque nenhum corpus de review tem timestamps.

O gerador foi escrito **contra** o vazamento que matou o primeiro fusor. A armadilha obvia seria "ironico = elogio + fato ruim", que ensinaria o modelo a procurar fato ruim em vez da incongruencia. Tres cruzamentos impedem isso, cada um travado por teste em `tests/test_gerador_ironia.py`:

- os mesmos **fatos ruins** aparecem nas reclamacoes diretas (nao-ironicas);
- as mesmas **palavras elogiosas** aparecem em elogios sinceros e em reclamacoes que comecam agradecendo;
- **numeros e unidades de tempo** aparecem nas duas classes;
- ha uma familia de ironia **por atenuacao**, sem palavra elogiosa nenhuma, para que "tem elogio" nao seja condicao necessaria.

A metrica interna sera otimista por construcao — ela mede o quanto o modelo aprendeu estes padroes, nao o quanto le ironia humana. Isso precisa estar no relatorio.

Por isso o `metricas_ironia.json` grava a **procedencia derivada de** `ORIGEM`, nunca `IDPT 2021` fixo: artefato que mente sobre a propria fonte e pior que artefato ausente.

Como o notebook **nunca viu os arquivos**, a celula de carga tem esquema **configuravel**: ela le todo `.csv`/`.tsv` da pasta, imprime as colunas encontradas, tenta achar a de texto e a de rotulo pelos nomes mais provaveis e **para nomeando as colunas disponiveis** se nao achar. Rotulo fora do mapeamento tambem para com erro, em vez de virar 0 silenciosamente.

Se o corpus nao for liberado a tempo, a saida honesta e declarar a ironia como trabalho futuro no relatorio. Trocar por um corpus de sarcasmo em ingles traduzido repetiria, com outro nome, o vazamento de procedencia que ja custou o primeiro fusor.

5.6.7 Portao de promocao do artefato

A acuracia/F1 do holdout sintetico **nao autoriza** substituir o modelo em producao. Depois do notebook 04, copie primeiro o checkpoint para uma pasta de candidato e rode:

```
uv run python scripts/validar_candidato_ironia.py /caminho/do/candidato \
  --saida-json metricas_regua_ironia.json
```

O comando exige no maximo 30% de falso positivo nas dez falas sinceras com marcador e 30% de falso negativo nas dez ironias sem marcador. Ele devolve codigo 1 se qualquer fatia falhar, para que o artefato nao seja promovido por uma unica media boa. As falas e o calculo canonicos moram em `fraus/avaliacao_ironia.py`; o laudo por fatias usa a mesma fonte.

Essa e uma **regua autoral de regressao**, deliberadamente desenhada contra os atalhos ja encontrados. Nao e corpus independente, nao estima acuracia em atendimento real e nao torna honesto reativar a ironia no fusor. Um corpus humano PT-BR separado do treino continua sendo o bloqueio cientifico.

A metrica de selecao do melhor checkpoint e o **F1 da classe ironica isolada**, nao a macro nem a acuracia: e essa classe que o sinal consome, e corpus de ironia costuma ser desequilibrado o bastante para um modelo que responde "nao e ironia" sempre marcar boa acuracia. O treino tambem usa pesos de classe, pelo mesmo motivo.

Por que uma cabeca separada e nao mais uma classe do sinal de texto: ironia nao e um sentimento, e uma relacao entre o que o texto DIZ e o que ele SIGNIFICA. "Que atendimento maravilhoso, so esperei 3 horas" e lexicamente positivo e pragmaticamente negativo ao mesmo tempo. Como quarta classe de satisfacao, o modelo seria obrigado a escolher uma das duas leituras — e a informacao de que ha conflito, que e justamente o sinal de ironia, se perderia.

Limitacao a declarar: o IDPT e anotado em tweets e comentarios de noticia, nao em atendimento de chatbot. A transferencia de dominio nao esta verificada — ironia em reclamacao de suporte pode ter forma diferente de ironia em comentario de politica.

5.7 SINAL LEXICO — SENTILEX-PT02

Nao tem notebook: e recurso pronto, nao treinado.

Fonte: SentiLex-PT02 — Silva, Carvalho e Sarmento, "*Building a Sentiment Lexicon for Social Judgement Mining*" (PROPOR 2012), CC-BY. 82.347 formas flexionadas na origem, **79.189** depois da conversao.

Preparo: `scripts/preparar_sentilex.py` converte o arquivo original em `fraus/dados/sentilex_pt02.csv` e imprime o relatorio do que mexeu — 315 entradas cuja polaridade veio do complemento (`POL:N1`), 9 com sinal normalizado (marcadas `REV:POL` na fonte, com o contador vazado no campo de polaridade) e 2.032 formas homografas com polaridades conflitantes, que foram **zeradas** em vez de decididas pela ordem do arquivo.

Por que um lexicon se o BERTimbau ja le o texto: porque ele nao foi treinado nos mesmos dados nem carrega os mesmos vieses. Dois sinais de procedencia independente que concordam sustentam a nota melhor que um sozinho — e a **discordancia** entre eles e o formato tipico da ironia.

Negacao por escopo: "nao foi otimo" inverte a polaridade de "otimo", com alcance de tres tokens, e o escopo **morre na pontuacao forte** — em "nao chegou. otimo atendimento" o *otimo* continua positivo.

Acento: o SentiLex e acentuado e cliente de chat nem sempre. Ha indice de reserva sem acento; das 74.443 chaves resultantes, apenas **24** colidem (`incomodo`/`incomodo`, `ingenua`/`ingenua`) e essas ficam zeradas — chute de polaridade errado e pior que termo ausente.

Limitacao do recurso, a declarar: o SentiLex e lexicon de **julgamento social** — anota polaridade dirigida a entidades humanas. E forte no adjetivo que julga (`pessimo`, `otimo`, `incompetente`) e **neutro em verbo de afeto do proprio falante**: `gostar`, `adorar` e `odiar` valem 0 nele. Por isso o sinal lexico complementa o BERTimbau e nao o substitui — quem le "adorei o produto" e o transformer.

5.8 SINAL DE ESTILO

Nao tem notebook: e **deterministico**, nao treinado — `fraus/sinais/estilo.py` mede a FORMA da escrita (caixa alta, pontuacao enfatica, alongamento, palavrao com gradacao, censura), nao o conteudo.

Por que deterministico e nao um classificador. O BERTimbau nao aprende enfase porque o B2W-Reviews01 — resenha moderada de e-commerce — praticamente nao contem gritaria nem xingamento. Modelo nao aprende

fenomeno que o corpus de treino nao tem, e mais epocas sobre o mesmo texto so reproduziriam o mesmo artefato. Regra escrita a mao cobre exatamente o que falta: forma, nao semantica.

Fonte do lexicon de palavrao: `fraus/dados/palavroes_ptbr.csv`, curadoria propria, ~172 termos, colunas `termo,intensidade,alvo`. A intensidade e graduada (`leve`/`medio`/`pesado`, mapeada para 0,33/0,66/1,0) porque "que droga" e "vai tomar no cu" nao sao o mesmo evento, e o alvo (`pessoa`/`desabafo`) existe porque xingar o PRODUTO e reclamacao enquanto xingar o ATENDENTE e ruptura da conversa. Nenhuma dependencia externa entrou para isso, e o SentiLex-PT02 tambem nao serve — ele anota polaridade de julgamento, nao gradacao de baixo calao nem alvo do xingamento.

Ordem censura-antes-de-normalizacao. `tem_censura` roda sobre a palavra CRUA, com a mascara (`*@#$%&`) ainda intacta, e so depois `normalizar` reverte homoglifo e apaga acento. Inverter a ordem apagaria a marca de censura antes de ela ser vista — "p*rra" perderia o asterisco e viraria uma palavra qualquer, nunca reconhecida como autocensura.

Casamento por curinga. `casar_censurado` trata cada simbolo de mascara como uma letra qualquer (regex com `.` no lugar do simbolo), nao como uma letra fixa: `p*rra` casa `porra`, `c*ralho` casa `caralho`. Mapear o simbolo para uma vogal fixa acertaria um caso e erraria o outro em silencio. Quando mais de um termo do lexicon casa o mesmo padrao com curinga, o desempate e pela MAIOR intensidade — deterministico, e nao pela ordem em que a linha aparece no CSV, o que deixaria o resultado dependente de onde alguem inseriu a linha.

A excecao do `kkkk`. Alongamento de caractere (`MINIMO_ALONGAMENTO = 3` repeticoes seguidas) e tratado como enfase, exceto para as letras de riso (`k`, `h`): "kkkk" e o marcador POSITIVO mais comum do chat brasileiro, e contar risada junto de "naooooo" inverteria o sentido da feature em boa parte das conversas reais.

Limitacao a declarar: o lexicon de palavrao e curadoria propria, nao recurso academico publicado. Diferente do SentiLex-PT02 (Silva, Carvalho e Sarmento, PROPOR 2012) e do Emoji Sentiment Ranking usados pelos outros sinais, `palavroes_ptbr.csv` nao tem paper citavel nem revisao por pares — a cobertura e a gradacao vieram de julgamento proprio, e isso precisa aparecer no relatorio como o que e.

5.9 CALIBRAÇÃO — A TEMPERATURA MORA NO ARTEFATO (DESDE 19/10/2026)

O antigo `notebooks/05_calibracao.ipynb` saiu em 19/10/2026. Ele ajustava a temperatura num conjunto "cego" que vazava para o treino da satisfação, e gravava `modelos/calibracao.json`, que nada no runtime lia.

Agora cada notebook de treino (01, 03, 04) ajusta a temperatura **na validação**, via `fraus.receita_treino.ajustar_temperatura`, e grava o valor em `fraus_receita.json` dentro da pasta do modelo. O runtime (`fraus/sinais/texto.py` e `fraus/sinais/onnx.py`) aplica essa temperatura; artefato sem o arquivo continua com `T = 1,0`, então os modelos em produção não mudam de score.

Temperature scaling não muda a decisão do modelo. Dividir os logits pelo mesmo `T > 0` preserva o `argmax`: acurácia e F1 ficam idênticos, só a confiança declarada muda (ECE).

A ironia é o caso a não confundir. Calibrar melhora o ECE dela, mas ela continua errando falas sinceras de atendimento. Temperatura ajusta *o quanto* o modelo se diz confiante, não *sobre o que*.

5.10 NOTEBOOK 02 — A SEÇÃO 10 É UMA PENDÊNCIA, NÃO UMA ETAPA

A saturação das features de tempo está descrita lá com a medição, o conserto (`winsorizar no p99` + `log1p` antes do scaler) e — o que mais importa — **o acoplamento**: a transformação precisa existir no notebook *e* no runtime, ou o score sai errado em silêncio. Um `fusor.joblib` treinado sem a compressão contra um runtime que comprime continua tendo `n_features_in_ = 39`, então `Fusor.carregar` **não pega** essa incompatibilidade.

5.10.1 As três saídas, medidas em 15/09/2026 — decidido: só `log1p`

O instrumento (hoje `scripts/retreinar_fusor_local.py`; a versão de medição está no commit `e719218`) reproduz este notebook localmente (mesmo B2W, mesmas sementes, mesmo `montar_features`, extração por ONNX) e **confere a reprodução antes de medir**: acurácia 0,9500 nos dois, score diferindo no máximo 0,0001 do `fusor.joblib` vigente. Nada foi promovido.

A coluna de espera é a frase canônica (texto 0,858 satisfeito, todo o resto na média do treino), com score, nota e categoria:

VARIANTE	ACURÁCIA	F1	CATEGORIAS ≠ VIGENTE (DE 300)	60 S	300 S	600
vigente	0,9500	0,9501	—	99,9 · 10 P	98,1 · 10 P	44,5
só `log1p`	0,9467	0,9469	6	99,7 · 10 P	96,6 · 10 P	90,1
só teto p99	0,9433	0,9434	1	99,9 · 10 P	96,1 · 10 P	18,4
p99 + `log1p`	0,9467	0,9469	5	99,7 · 10 P	96,6 · 10 P	90,7

O que a tabela muda na recomendação escrita acima:

- **p99 + `log1p` conserta o relógio mandando sozinho apagando a espera longa.** O teto do treino é ~7 min (`latencia_mediana_s` p99 = 424 s), então três horas pesam o mesmo que dez minutos: a frase canônica, que é **irônica**, sai promotor. Consertar um defeito criando o oposto não é conserto.
- **Só teto** não resolve nada: o penhasco só muda de lugar (10 min → nota 2).
- **Só `log1p`** é a única que mantém a espera **monótona e sem penhasco**: 10 min ainda promotor, 30 min detrator 6, 3 h detrator 1 — o texto modula, o relógio pesa cada vez mais devagar, e nenhum dos dois apaga o outro. Custa 0,33 ponto de acurácia no teste sintético.
- **Ablação da invariante 10:** só as quatro features de tempo acertam 0,513 (cru) e 0,527 (p99+log1p), contra 0,333 do acaso. O relógio carrega sinal do rótulo no simulador por construção, e nenhuma das variantes muda isso.

5.10.2 Promovido em 15/09/2026

As três partes indivisíveis, feitas juntas:

1. **Função única.** `log1p` mora em `fraus.fusor.vetorizar`, sobre `FEATURES_EM_LOG`. As chaves continuam em segundos (atribuição, tela e deriva leem segundos); só o vetor que o scaler recebe é comprimido. Treino, pontuação, contribuição e deriva passam pela mesma função.
2. **Artefato regerado** por `scripts/retreinar_fusor_local.py --promover`: acurácia 0,9467, F1-macro 0,9469. O anterior ficou em `modelos/fusor.joblib.bak-segundos`. O artefato agora carrega `fraus_escala_do_tempo = "log1p"`, e `Fusor.carregar` **recusa** artefato sem essa marca: é o caso que `n_features_in_` não pega.
3. **Medição de categoria.** No teste do B2W, 27 notas e 6 de 300 categorias mudam. Nas 180 conversas do simulador, 23 notas e 7 categorias, com a distribuição indo de 60/63/57 para 60/60/60 (detrator/neutro/promotor).

`scripts/medir_dominio_do_tempo.py` contra o artefato novo: o relógio empata com o texto em **613 s** (antes 411 s), e com 3 h pesa **2,1×** o texto (antes 20×). Uma conversa sincera ("resolveu, muito obrigado") com 30 min de espera saía 0,30; agora sai 55,3, ainda detrator. Com 3 h, 21,95 em vez de 0,00.

Quem tem o artefato em outro lugar precisa regerar (a VM, o Drive): um `fusor.joblib` de antes de 15/09 não sobe mais, e isso é o comportamento correto da invariante 7.

6. Ironia: corpus e Laya

> **Estado em 08/10/2026.** O BERTimbau de ironia foi retreinado no notebook 04 com a divisão por autor corrigida (treino 28.365 linhas, 58,8% irônicas; teste 4.778, 59,2%). Deu F1-macro de **0,994** no teste interno (0,991 só nos tweets) e P(irônico) = **1,000** em frases neutras de atendimento, como "qual o prazo de entrega do meu pedido" e "ok, obrigado". O modelo **não foi exportado** e o Laya não foi treinado. Esta página explica por quê e o que fazer antes do próximo treino. Medições e passos da sessão: nota interna `docs/notas/2026-10-08-ironia-corpus.md`.
>
> **Produção não muda:** a ironia em produção é lida pelo Laya sem treino (`laya-onnx`) e não entra no vetor do Fusor.

6.1 RESUMO: CONSERTAR O CORPUS ANTES DE TREINAR O LAYA

Treinar o Laya (mmBERT-base, `laya==0.3.21`) só em ironia é fácil de fazer e não resolve o problema que motivou a ideia. O vício do IDPT 2021 está nos dados, não no modelo nem na mistura de tarefas: a classe não irônica dos tweets é **100% de tweets com #economia**, publicados por ~658 contas, quase todas portais de notícia, só em maio e junho de 2017 (arquivo bruto). As notícias foram rotuladas **pelo site de origem**, "sem nenhuma rotulação manual" (Marten & de Freitas, ENIAC 2020). Qualquer encoder afinado nesse rótulo aprende registro e procedência, seja o Laya sozinho, o Laya multitarefa ou o BERTimbau. O formato de pergunta do Laya não protege contra isso, porque a instrução e os critérios são tokens idênticos em todos os exemplos e o gradiente vem só do texto. Chegar a "100% de exatidão" nesse teste não seria a prova esperada. Pela invariante 10 do projeto, seria o sintoma: ironia é uma tarefa em que anotadores humanos concordam pouco, e um acerto perfeito mostra que o teste mede a fonte. O ganho real virá de três coisas: trocar o negativo por fala coloquial sincera, medir numa régua de domínio com 200 a 400 itens em pares mínimos e rodar três treinos que separem as causas. Pela literatura, a diferença de encoder entre o Laya e o BERTimbau deve ficar em poucos pontos. Contra o multitarefa de 02/10, o Laya só-ironia quase certamente vai "ganhar" no teste interno, mas esse ganho virá da divisão corrigida, e não do fato de treinar só ironia.

6.2 O IDPT ENTREGA O RÓTULO, E NENHUM ENCODER ESCAPA DISSO

O achado de 08/10 confirma o que os próprios arquivos mostram. No `nao-ironico.csv`, o commit `b735dd9` que o Fraus fixa, as hashtags dominantes são **2.519** `#economia` **e 185** `#econom`, e as datas vão só de 2017-05 a 2017-06. As contas mais frequentes são `noticiaminutobr`, `TSFRadio` (rádio portuguesa) e `portalR7`, e há tweets em espanhol e em português europeu no meio. Do outro lado, o `ironia.csv` tem ~12.700 tweets auto-marcados com `#ironia`, de ~9.300 autores, publicados entre 2014 e 2017. Só 78 tweets irônicos falam de economia, contra ~2.564 não irônicos (dados verificados). O `scripts/preparar_corpus_ironia.py` remove a hashtag `#ironia`, mas o vazamento continua no tópico, na fonte e na época. Nas notícias, `estadao.json` é sempre 0, e `sensacionalista.json` e `the_piaui_herald.json` são sempre 1. Ou seja, a tabela fonte × rótulo tem **células vazias**: nenhum tweet coloquial sincero e nenhuma notícia séria irônica.

O histórico público do IDPT já mostrava isso antes do Fraus. O treino oficial tem 15.212 tweets e 18.494 notícias, enquanto os testes são "300 tweets and 300 news gathered and annotated by the organizers" (García Subies, CEUR Vol-2943). No teste de tweets, anotado por humanos, **o melhor time chegou a 0,52 de acurácia balanceada** (Luz et al., BraSNAM 2023), e o BERTimbau afinado ficou em 0,49–0,50, abaixo do acaso (García Subies). Enquanto isso, o SiDi-NLP relatou **F1 1,000 na validação de tweets**, tanto com BERTimbau quanto com mBERT (CEUR Vol-2943). O F1 de 0,99 que o BERTimbau retreinado deu no teste interno do Fraus, junto com P(irônico)=1,0 em frases neutras de atendimento, repete esse padrão: um número alto dentro da distribuição que desaba quando o rótulo vem de um humano. O padrão é conhecido na literatura. Quando o rótulo de sarcasmo passa a ser dado pelo próprio autor, o melhor modelo cai de F1 0,874 para 0,364 (Oprea & Magdy, ACL 2020). Filtrar os vieses do SNLI derruba um modelo de 92% para 62% (Le Bras et al., 2020).

A esperança de que o Laya "evitaria dados viciados" esbarra em como ele aprende. O fine-tuning do script oficial é completo, com encoder e cabeça juntos, sem LoRA. A sequência sempre começa com a mesma instrução e os mesmos critérios A/B de `PERGUNTA_IRONIA` (`laya/common.py`, `build_sequence`). O que separa A de B é só o `state`, isto é, o texto. Se registro, língua ou gênero textual predizem o rótulo, o mmBERT aprende isso igualzinho ao BERTimbau. O próprio guia do Laya avisa: "The loop is only as good as the targets… treat their quality as the ceiling" (finetune.md). O único ponto a favor do Laya tinha sido o prior zero-shot. Sem treino, ele fez F1 0,554 no teste interno e 0,524 na régua, com 0% de falso positivo, melhor que o BERTimbau. O run de 02/10 mostrou que o fine-tuning completo apaga esse prior (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`).

O "100% de exatidão" é inalcançável por um motivo independente do corpus: ironia é ambígua até para humanos. No Tweets8JanSPI, anotado à mão, o Fleiss κ da ironia foi **0,38**, contra 0,59 do sentimento (BraSNAM 2026). O MultiPICo usa em média cinco anotadores por item justamente porque o julgamento varia (ACL 2024). Um modelo não pode concordar com um rótulo mais do que os humanos concordam entre si, a menos que esteja lendo outra coisa. A invariante 10 diz isso com outras palavras: "acurácia alta demais é sintoma, não vitória".

6.3 TREINAR SÓ IRONIA EXIGE MEXER EM SETE CÉLULAS E NUM RÓTULO DE PRODUÇÃO

O pacote `laya==0.3.21` só faz inferência e não tem API de treino. O fine-tuning roda pelo script de pesquisa `finetune_single_device.py`, no commit `d8a2e597` (script). O notebook 07 já o executa em subprocesso e confere o SHA-256. O script aceita tarefa única direto: basta o JSONL ter apenas linhas de `caso_ironia(texto, p)`, com `gold.ironia.probabilities = {"A": p, "B": 1-p}`, em que A = irônico. A ordem das chaves de `criteria` define a ordem do alvo. Por isso `PERGUNTA_IRONIA` tem de ser importada de `fraus/sinais/ironia_laya.py` e nunca redigitada, que é o mesmo cuidado que a invariante 8 pede para a ordem das classes.

As mudanças no notebook 07 (`notebooks/07_treino_laya.ipynb`) são estas, pela numeração "#" dos comentários das células:

CÉLULA	HOJE	PARA SÓ IRONIA
#2 (imports)	importa emoção e ironia	pode ficar; os nomes de emoção deixam de ser usados
#4 (corpus de emoção)	GoEmotions-pt + XED-pt	**pular inteira**; define `dados_emocao`, `emocao_teste`, `frases_xed`, usados adiante
#5 (divisão de ironia)	`dividir_por_autor`, `conferir_divisao`, guarda de impressão contra `metricas_ironia.json`	**não mexer**: é o que garante que o BERTimbau do notebook 04 foi treinado na mesma divisão
#6 (JSONL)	laço de emoção com `repeticoes_por_classe` + ironia suavizada 0,05	só ironia; `proibidos` = teste de ironia ∪ régua; manter o `assert` de vazamento
#7 (treino)	`MODO_FUMACA`, `EPOCAS=4`	fumaça primeiro (600 casos), depois completo
#8 (comparação)	confere `DIR_BERT_EMOCAO`, prevê emoção e XED	tirar a emoção; manter `prever_*`, `comparar`, `laya.load`
#9 (ironia)	teste interno, recortes `tweets`/ `noticias`, régua	sem mudança
#10 (poda + ONNX)	vocabulário com emoção, `cobertura_xed`	`perguntas = {**PERGUNTA_IRONIA}`; a cobertura precisa de outro texto independente
#11 (manifesto)	`perguntas: ["emocao","ironia"]`	`["ironia"]`; **ver a armadilha abaixo**
#12 (laudo)	conjuntos de emoção e ironia	só `ironia_interno`, `ironia_tweets`, `ironia_noticias`, `ironia_regua`

A **armadilha do manifesto** é a parte que não aparece no notebook. Em `fraus/sinais/emocao_laya.py`, `laya_treinado_pelo_fraus(diretorio)` devolve `"emocao" in perguntas_do_artefato(diretorio)`, e um manifesto ausente equivale a `("ironia",)`. Assim, um artefato treinado só em ironia é **indistinguível do checkpoint base**, e a dashboard o rotularia como "Laya sem treino", o que é falso. Há um segundo efeito, mais silencioso: `obter_classificador_emocao_laya_onnx` reaproveita o mesmo agente e responderia emoção com uma cabeça afinada só em ironia. A correção precisa vir antes do treino e com TDD. O manifesto ganha um marcador explícito de procedência, por exemplo `"treinado_pelo_fraus": true` junto com a revisão e `rodada_de_fumaca: false`. `laya_treinado_pelo_fraus` passa a ler esse campo, e a leitura de emoção pelo Laya passa a recusar, com falha alta, um artefato que não declara `emocao`. Os dois testes falham primeiro com o manifesto só-ironia e passam depois.

Os hiperparâmetros estão quase todos fixos no código do script. A linha de comando aceita só `--data`, `--model-dir`, `--output-dir`, `--device`, `--epochs` e `--seed`. Mudar qualquer outra coisa obriga a editar o arquivo, o que quebra o SHA-256 conferido e precisa ser declarado como desvio do caminho oficial.

PARÂMETRO	VALOR NO SCRIPT	RECOMENDAÇÃO PARA SÓ IRONIA
Micro-lote / acumulação	8 / nenhuma (lote efetivo 8)	manter; a receita 2×T4 do autor usa lote efetivo 64, mas isso exige editar o script
LR encoder / cabeça	2,5e-5 / 1e-4, AdamW wd 0,01	manter
Agenda	cosseno até 1e-6, sem warmup; σ do ruído de 0,4 a 0,1	manter
Precisão	fp16 autocast + gradient checkpointing, clip 1,0	manter
`max_len`	1024 (forçado)	controlar pelo `MAX_CARACTERES` do notebook: **280–400** em vez de 1200, para aproximar o tamanho de uma mensagem de chat e baratear o treino
Épocas	padrão 4	**2 a 3**: o corpus tem um atalho forte, e mais épocas tendem a decorá-lo
Semente	padrão 0	42 e mais duas sementes se o tempo couber, porque o fine-tuning é instável entre sementes (Dodge et al. 2020; Mosbach et al. 2021)
Peso de classe	inexistente	dispensável: a divisão corrigida tem 59% de irônicas dos dois lados (≈1,4:1)
Suavização do alvo	0,05	considerar 0,1: rótulo por procedência não sustenta certeza de 0,975
Calibração	temperatura por LBFGS em `min(400, n//10)` itens separados antes do treino	manter; ela não muda o argmax, mas muda qualquer limiar de 0,5

Sobre o tempo na T4: o run de 02/10 processou 142.204 itens (71.102 × 2 épocas) em 89 minutos, cerca de 26,6 itens/s com a carga incluída (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`). Com os 28.365 itens de treino da divisão corrigida, a conta direta dá **~18 minutos por época, ~35 minutos para 2 épocas e ~70–75 minutos para 4**. Essa é uma extrapolação de um único run. As notícias são bem mais longas que as frases do GoEmotions que dominavam aquele treino, então o tempo real pode ser 1,5 a 2× maior. A estimativa que vale é a que a rodada de fumaça imprime. O script não salva checkpoint por época. Chamá-lo várias vezes com `--epochs 1` também não equivale a um treino de N épocas, porque reinicia o cosseno, a σ do ruído e o otimizador e separa de novo a fatia de calibração. Como 2 a 3 épocas cabem numa sessão T4, o caminho mais simples é o atual: copiar para o Drive no fim.

6.4 CONSERTAR O NEGATIVO VALE MAIS QUE TROCAR O ENCODER

Os métodos de debiasing que mexem no modelo, como product-of-experts, autoviés, group DRO e DANN, só redistribuem peso entre exemplos que **contradizem** o atalho (Clark et al. 2019; Sagawa et al. 2020). Com as células "coloquial × não irônico" e "notícia séria × irônico" vazias, não existe exemplo conflitante para repesar. Se a fonte é quase igual ao rótulo, apagar a fonte por treino adversarial apaga o rótulo junto. A correção tem de ser nos dados. O primeiro passo custa minutos: imprimir a tabela de contingência fonte × rótulo e, além dela, treinar um classificador "adivinhe a fonte". Se o rótulo sai da fonte, o corpus reprova antes de qualquer GPU, do mesmo jeito que a invariante 10 manda checar a sobreposição de distribuições por rótulo.

Não existe hoje um corpus PT-BR de ironia que seja grande, anotado por humanos e com negativos coloquiais naturais. O melhor candidato é o **MultiPICo-PT**: 1.994 pares post-resposta de Reddit e Twitter, metade brasileira e metade europeia, 49 anotadores nativos, ~38% irônicos, licença CC BY 4.0 e acesso livre no Hugging Face (MultiPICo, ACL 2024; HF). As duas classes saem da mesma amostragem e o rótulo vem do julgamento humano, não da fonte. O formato post-resposta combina com o Fraus, em que a fala do bot vem antes da do cliente. O Tweets8JanSPI (2.935 tweets, 224 irônicos, κ 0,38) tem o mesmo desenho, mas é distribuído desidratado, só com IDs, e trata de um único tópico (BraSNAM 2026). O Kaggle "viniciuscleves" é um clone do mesmo corpus do IDPT (Kaggle). O de manchetes The Onion × HuffPost reproduz o rótulo por procedência (Misra & Arora).

A receita recomendada tem quatro camadas, da mais à menos importante. A primeira é **descartar as notícias**, porque nenhum dos dois lados é fala de cliente, e **substituir o** `nao-ironico.csv` por negativos coloquiais estratificados por polaridade: muito elogio sincero e muita reclamação literal, por exemplo B2W com 5 e com 1 estrela (licença CC BY-NC-SA 4.0, compatível com uso acadêmico (README B2W)) e frases do GoEmotions-PT, que é Apache 2.0. Isso ataca o modo de falha já medido: a cabeça atual marca 74% das resenhas positivas da B2W como irônicas, contra 9% das negativas (`docs/treinamento.md`, 04/09/2026). A segunda é **somar o MultiPICo-PT**, como fatia de treino e de validação. A terceira são os **pares mínimos escritos à mão** no domínio de atendimento, no molde da subtarefa pareada do iSarcasmEval (Abu Farha et al. 2022) e dos negativos por micro-edição do Puntuguese (HF). Um exemplo de par: "ótimo serviço, resolveram rápido" × "ótimo serviço, só esperei 3 horas". Esses pares rendem mais como teste do que como treino, porque dados contrafactuais protegem as features perturbadas, mas podem piorar as que não foram perturbadas (Joshi & He 2022). A quarta é **minerar positivos de domínio** com heurística (elogio + situação negativa, "só que não", "parabéns" numa nota 1) e **confirmar cada um à mão**. Se a heurística virar o rótulo, o projeto volta ao rótulo por marcador. Os tweets irônicos do IDPT podem continuar como positivos, desde que tenham negativos do mesmo meio. Normalizar o estilo (minúsculas, sem URL, @, hashtag nem ponto final) é barato e remove pistas tipográficas, mas não remove o léxico de registro ("PIB", "Meirelles"). Por isso o efeito tem de ser medido com o baseline de estilo, antes e depois.

6.5 TRÊS TREINOS E UMA RÉGUA DE 300 PARES SEPARAM AS CAUSAS

O colapso de 02/10 tem uma explicação demonstrada que não depende da multitarefa. A divisão saiu com treino 86% irônico e teste 43% irônico, porque as 18.373 notícias sem autor caíam todas no teste. O Laya treinado chegou a recall de **0,988 no irônico e 0,087 no não irônico**, com F1-macro de 0,388. A mesma divisão fez o BERTimbau ficar em 0,511, e a rodada de fumaça já mostrava 100% de falso positivo (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`). Ainda assim, a hipótese de interferência entre tarefas tem base. No pre-

finetuning multitarefa, o desempenho cai quando há poucas tarefas (Muppet), e a mistura de 02/10 era 83% emoção para 17% ironia. Mas emoção e sentimento como tarefas auxiliares **ajudam** o sarcasmo em outros trabalhos (Chauhan et al. 2020). Só um desenho controlado decide.

TREINO	O QUE VARIA	COMPARAÇÃO QUE RESPONDE
(i) Laya só ironia	tarefa única, divisão corrigida	(i) × (ii): a multitarefa atrapalha?
(ii) Laya emoção + ironia **balanceado**	amostragem por temperatura ou uma época de ironia por época de emoção	(i) × (ii): se Δ ≈ 0, a hipótese da multitarefa cai
(iii) BERTimbau só ironia (notebook 04)	encoder	(i) × (iii): o encoder importa, com dado e tarefa iguais?

Os três treinos usam a mesma divisão, conferida pela impressão de `metricas_ironia.json`, e a mesma semente. Cada um deve rodar duas vezes: uma no IDPT corrigido e outra no corpus consertado da seção anterior. A troca de corpus é o fator que a literatura indica como dominante, e esse fator 2 × 3 é o que a banca vai querer ver. A divisão por autor não basta para medir o atalho. Hoje cada notícia sem autor vira um grupo próprio e se espalha pelos dois lados. Por isso é preciso uma função irmã de `dividir_por_autor` que agrupe por **fonte** (leave-one-source-out) e reaproveite `conferir_divisao`. Essa função vai falhar alto se a fonte for quase igual ao rótulo, e essa falha já é o diagnóstico.

O teste interno não decide nada, já que o BERTimbau marca 0,99 nele. Quem decide é uma **régua de domínio independente**, escrita por quem não viu as predições, congelada com `impressao_dos_textos` antes da primeira avaliação e marcada `independente=True` no laudo. A régua tem **200 a 400 itens em pares mínimos** e quatro estratos: elogio sincero, elogio irônico, reclamação literal (que separa ironia de sentimento negativo) e ironia sem marcador. Inclui ainda testes de invariância no estilo do CheckList (Ribeiro et al. 2020): o mesmo texto com e sem ponto final, em caixa alta ou baixa e com um emoji neutro tem de receber a mesma predição. A métrica principal é a **acurácia por par**, em que o modelo acerta os dois membros. O acaso aqui é 25%, e um modelo de estilo não consegue acertar os dois lados de um par com o mesmo registro. O tamanho sai de uma conta binomial própria: com acurácia em torno de 0,8, n=100 dá IC de ±7,8 pontos, n=200 dá ±5,5 e n=400 dá ±3,9. A régua atual tem 20 itens e IC de −0,28 a +0,17, larga demais para decidir qualquer coisa. Pares mínimos com troca de rótulo derrubam modelos em até 25% em relação ao teste original (Gardner et al. 2020), e é essa queda que o Fraus precisa conseguir medir.

Três baselines entram como modelos extras em `preditos_por_modelo`, no espírito do baseline "só a hipótese", que acerta ~67% no SNLI (Gururangan et al. 2018). O primeiro é **só estilo**: regressão logística sobre ~15 features sem léxico (comprimento, proporção de maiúsculas, ponto final, `!?...`, dígitos e R$, URL, @, hashtag, emoji, alongamento, "kkk"). O segundo é **só fonte**. O terceiro é **TF-IDF + LR**, que mostra a parte lexical do registro. Se o baseline de estilo também chega perto de 0,99 no teste interno, o teste mede procedência. O modelo só conta como detector de ironia se vencer o baseline de estilo **na régua**, com diferença demonstrada. `fraus/comparacao_modelos.py` já tem `mcnemar_exato`, `bootstrap_da_diferenca` (2.000 reamostras, F1-macro), `avaliar_conjunto` e `montar_laudo`, que já distinguem procedência igual de independente. As quatro peças que faltavam já existem no mesmo módulo: IC de um único modelo, **bootstrap por par** (reamostrar pares, e não itens, para preservar a dependência dentro do par), acurácia por par e **ECE** com diagrama de confiabilidade (Guo et al. 2017). O ECE importa porque o Laya sai superconfiante e a temperatura dele é ajustada numa fatia do próprio corpus viciado.

Para o McNemar, o que conta são os pares discordantes: com ~20 discordantes, só uma divisão de 15:5 ou mais dá p<0,05. Como guarda de regressão, entra a medição que já existe em `docs/treinamento.md`: P(irônico) médio por estrela da B2W. Se a diferença entre resenhas positivas e negativas continuar grande, o atalho continua lá.

6.6 O GANHO ESPERADO É DE POUCOS PONTOS, E VEM DO DADO

Contra o multitarefa de 02/10, o Laya só-ironia vai melhorar muito no teste interno: de 0,388 para algo bem acima, talvez perto do 0,99 do BERTimbau. Esse salto vem da divisão corrigida e da proporção de 59/59, que eliminam o colapso "tudo é irônico". Ele não deve ser atribuído ao treino em tarefa única nem apresentado como detecção de ironia. Na comparação justa, o treino (i) contra o (ii) balanceado, o tamanho de efeito publicado para multitarefa é de **poucos pontos, em qualquer direção** (Standley et al. 2020; Chauhan et al. 2020). O que se espera é empate dentro do IC.

Contra o BERTimbau, com tarefa e dado iguais, a literatura não dá motivo para esperar diferença grande. Em PT-BR, o BERTimbau vence o mBERT por margens de 2 a 4 pontos, por exemplo 89,2 contra 86,8 de F1 no ASSIN2 RTE (neuralmind-ai). O mmBERT-base supera o XLM-R base no XTREME (72,8 contra 70,4), mas português é só ~2,4% da mistura de pré-treino, e o artigo não traz nenhum benchmark de classificação em português (Marone et al. 2025). Há um precedente em que o mmBERT-base venceu o BERTimbau em NER clínico em português, com micro-F1 de 0,76, num preprint ainda sem revisão (arXiv 2603.26510). Não existe resultado de mmBERT, XLM-R ou Laya no IDPT, nem de afinação do Laya em ironia em qualquer língua. Na CPU, o Laya é mais lento: 459 ms contra 273 ms por mensagem na ironia, com ressalva, porque o Laya rodou em ONNX FP32 e o BERTimbau em PyTorch (nota interna de 02/10, `docs/notas/2026-10-02-treino-laya.md`). A velocidade do ModernBERT só aparece em GPU e em sequências longas.

A única vantagem estrutural do Laya para ironia é o vocabulário multilíngue, que permite misturar o MultiPICo das outras oito línguas. A literatura, porém, é desfavorável à transferência de ironia entre línguas: o viés de tópico dos corpora atrapalha (Ortega-Bueno et al. 2023), e detectores de sarcasmo não generalizam entre datasets (Jang & Frassinelli 2024). O ganho que de fato muda o produto aparece na régua de domínio, quando se troca o corpus. Um modelo treinado no IDPT deve ficar perto do acaso por par, como o 0,52 oficial nos tweets. Um modelo treinado com negativos coloquiais e pares mínimos tem espaço para ganhar dezenas de pontos ali, na escala das quedas de 25 a 30 pontos que a literatura de contrast sets e filtragem adversarial atribui ao viés. Essa magnitude é inferência por analogia e não foi medida em português. A régua de 300 pares existe justamente para medi-la.

6.7 RÉGUA DE PARES MÍNIMOS

Estado: **rascunho pronto, anotação em andamento.** Spec em `docs/superpowers/specs/2026-10-08-regua-pares-ironia-design.md` (interno).

- 150 pares (elogio, reclamação, neutra), registro equilibrado conferido por `conferir_registro_equilibrado` e, par a par, mesma vírgula e mesmo número de "?" nos dois lados. O baseline só de estilo acerta 0,18 dos pares fora da amostra no rascunho, abaixo do acaso (0,25). O rascunho é autoral; o rótulo que vale é o da anotação às cegas.
- A anotação mora no Fraus, por link e sem conta: `scripts/criar_link_anotacao.py --quantos N` chama `POST /anotacao/anotadores` (só dev) e imprime `<id>  https://fraus-one.vercel.app/anotar/<token>`. O banco guarda só o hash do token e um id opaco (`a_` + 8 hex), nunca nome. A página recebe de `GET /anotacao/<token>` só `id` e `texto`, na ordem daquele anotador, e grava por `POST /anotacao/<token>/respostas`; o instante é do servidor. A primeira rodada (João, na página antiga do claude.ai) está versionada em `fraus/dados/anotacao_regua/anotador-claude-ai.json`, anotador `claude-ai-1`.
- Para um grupo de mensagens, `scripts/criar_link_anotacao.py --grupo 5` cria **um** link (`https://fraus-one.vercel.app/anotar/grupo/<token>`) com 5 vagas. Cada pessoa que o abre ganha o próprio anotador (a vaga é ocupada de forma serializada no banco, então 5 não viram 6) e a página guarda o link pessoal só naquele navegador. Em outro aparelho, abrir o link do grupo de novo gasta outra vaga e começa do zero; quem quiser trocar de aparelho guarda o endereço da página pessoal. Repassar o link do grupo também gasta vaga. Grupo cheio responde "Este link já foi usado por todas as pessoas previstas". Revogar o grupo (`POST /anotacao/grupos/<g_id>/revogar`) fecha a entrada; com `?anotadores=1` revoga também quem entrou, e as respostas saem do export.
- O token viaja na URL e fica nos logs de requisição da Vercel e no histórico do navegador, como o link de convite. Link vazado se revoga com `scripts/criar_link_anotacao.py --revogar <id>`; as respostas desse anotador saem do export.
- **Critério de promoção de qualquer cabeça de ironia:** vencer o baseline só de estilo (`fraus/baseline_estilo.py`) em acurácia por par na régua, com IC 95% da diferença excluindo zero. O teste interno do IDPT não entra. No laudo isso é o campo `candidato_vence_por_par` (limite inferior do IC da diferença acima de zero). `por_par_demonstrada` só diz que o IC exclui zero e também acende quando o candidato é significativamente pior; para promover, vale `candidato_vence_por_par`.
- Depois da anotação: `uv run python scripts/exportar_anotacao.py respostas-fraus.json` baixa `GET /anotacao/respostas` (só dev) e `uv run python scripts/consolidar_regua_ironia.py fraus/dados/anotacao_regua/anotador-claude-ai.json respostas-fraus.json` concatena os arquivos e consolida. Menos de 100 pares confirmados para o script com erro dizendo quantos faltam.
- Cada registro é `{"anotador": "...", "frase_id": "...", "resposta": "ironico|nao_ironico|contexto", "instante": "2026-10-08T12:00:00.000000+00:00"}`; o script usa a última resposta de cada anotador por frase.
- A página embaralha a ordem por anotador (semente derivada do id) e nunca põe os dois lados de um par em sequência. A régua congelada guarda duas impressões no meta: uma dos textos e outra de `par_id|rótulo|texto`, de modo que trocar um rótulo ou mover um par também falha alto na carga.

6.8 CONCLUSÃO

A pergunta "Laya ou BERTimbau, só ironia ou multitarefa" foi formulada sobre um corpus que não sustenta nenhuma das quatro respostas, porque nele o rótulo é a fonte. A decisão que de fato move o resultado vem antes do encoder. São três passos: trocar o `nao-ironico.csv` de #economia e as notícias por fala coloquial sincera,

construir a régua de pares mínimos e comparar cada modelo com um baseline de estilo que não sabe nada de ironia. O Laya só-ironia continua valendo como experimento, como a célula (i) de um desenho controlado e não como conserto, e só depois de corrigir `laya_treinado_pelo_fraus` para que o produto não rotule um modelo treinado como "sem treino".

Para a banca, a meta de 100% se transforma num argumento a favor do projeto. Mostrar que o BERTimbau marca 0,99 no teste interno, que um classificador de pontuação e caixa alta chega perto disso, e que ambos caem para perto do acaso numa régua de pares mínimos é uma demonstração empírica da invariante 10 em ironia. Essa demonstração convence mais do que qualquer número alto. Promover o modelo depende de ele vencer o baseline de estilo na régua independente, com IC que exclua zero. O teste interno não serve para essa decisão.

7. Integração com o Tars

Os projetos continuam independentes. O Tars é o **produtor** das conversas e o Fraus é o **consumidor** analítico; nenhum acessa o banco ou importa código do outro. O único contrato entre os dois é `POST /ingestao`, a mesma rota que qualquer sistema externo usa.

Em produção desde 19/10/2026 (Fraus #97 e #98; Tars #28, #29 e #30).

7.1 CONECTAR, EM DOIS PASSOS

1. **No Fraus**, menu da conta → **Integrações** → cartão **TARS** → **Conectar o TARS**. A tela cria a fonte de canal `tars`, gera a chave `frs_...` (mostrada **uma única vez**) e mostra o endereço `https://fraus-api.vercel.app/ingestao`.
2. **No Tars**, **Integrações → Fraus** do assistente: cole o endereço e a chave e clique em **Conectar e enviar**. O Tars guarda a chave cifrada (como o token do Zendesk) e já manda as conversas da última semana — é o teste de verdade, porque só o Fraus sabe se a chave vale.

Gerar chave e segredo de fonte exige conta `dev` no Fraus (`exigir_mestra_ou_dev`, em `fraus/api/seguranca.py`). Antes de 19/10 a rota exigia a chave mestra, que o proxy da dashboard nunca manda — o botão dava 403 para o próprio desenvolvedor. A chave de **acesso** continua só com a mestra.

7.2 O QUE É UM ATENDIMENTO

Um chat do Telegram dura semanas; um atendimento, não. **30 minutos sem fala encerram o atendimento**, e a próxima mensagem abre outro, com a data dela (`dividirEmSessoes`, em `lib/integracoes/fraus-pedido.mjs` do Tars).

O id que chega ao Fraus é `tars:<instância>:<canal>:<chat>:<início>`, e o Fraus ainda o prefixa com a fonte (`fonte:<id>:...`). O mesmo id **substitui** a versão anterior (upsert), então reenviar nunca duplica.

O Tars poda o histórico em 40 mensagens por chat. Recalcular o id pelo primeiro horário que sobrou criaria um atendimento novo a cada poda; por isso o id do atendimento em curso fica guardado em `fraus_envios.sessao_id` e é herdado enquanto a conversa não pausa 30 minutos.

7.3 QUANDO O ENVIO ACONTECE

Depois de cada resposta gravada (`registrarResposta`/`registrarTurno`), o Tars agenda o envio com `after()` — **nunca atrasa a resposta ao cliente**. O envio espera **5 segundos** e só segue se nenhuma fala mais nova chegou; se

chegou, o envio da fala mais nova é que manda. Uma rajada de mensagens vira um envio só, e a última versão do atendimento sempre chega.

> Até 19/10, 17h30, o Tars usava um freio de um envio por chat por minuto. Duas mensagens no mesmo minuto deixavam a última fala para o cron do dia seguinte: o atendimento terminava em "péssimo" e o Fraus o mostrava como neutro.

A **repescagem** manda o que falhou: o botão **Enviar conversas pendentes** e o cron diário do Tars, que só reenviam atendimentos com fala depois do último envio. Na 10ª falha seguida o envio desliga e o operador recebe um alerta; salvar a conexão de novo religa.

7.4 O PEDIDO

```
{
  "id": "tars:inst_x:telegram:8624457065:1792441687",
  "mensagens": [
    {"autor": "cliente", "texto": "...", "enviada_em": "2026-10-19T20:28:07.000Z"},
    {"autor": "bot", "texto": "...", "enviada_em": "2026-10-19T20:28:09.000Z"}
  ],
  "escalou_para_humano": false
}
```

- `user` vira `cliente`; `assistant` vira `bot`; resposta escrita pela equipe no painel vira `humano` e marca `escalou_para_humano`.
- Horário em ISO com fuso — o Fraus recusa data sem fuso (invariante 6).
- O corpo cabe no teto de 32 KB do `chamarApiSegura`: se não couber, só as respostas longas do bot são encurtadas; a fala do cliente nunca é cortada.
- Score, nota e categoria **nunca** vão no pedido: o Fraus deriva no servidor (invariante 3). O canal vem da fonte cadastrada, não do corpo.

7.5 PRIVACIDADE — O QUE VALE E O QUE FALTA

- O Fraus aplica a censura de PII ao texto antes de persistir.
- **O id leva o `chat_id` do Telegram em claro.** É um identificador de conta, não conteúdo, mas atravessa a fronteira. Trocar por um hash estável (com sal da instância) é a próxima mudança de privacidade; como o id novo não colide com o antigo, a troca pede remover antes os atendimentos do Tars no Fraus.
- Os botões "Resolveu / Não resolveu" do Tars ainda **não** são enviados como `feedback_declarado`.

7.6 LIMPEZA

Atendimentos que nunca deveriam ter entrado (o datado de 2243, duplicatas de teste, o chat inteiro no formato anterior a 19/10) saem pelo botão **Remover atendimento**, no detalhe do atendimento — só conta `dev`, com confirmação, e registrado na auditoria (`DELETE /conversas/{id}`).

name: Fraus — Contexto, não certeza description: Grafite, monograma FR em camadas e tipografia quadrada. Uma chegada expressiva e uma ferramenta de leitura calma, com temas escuro e claro. colors: background: "#171819" foreground: "#F3F4F5" card: "#222426" muted-foreground: "#B3B9C0" border: "#454B52" primary: "#D6BA72" dito: "#EDB466" medido: "#8DC8F5" typography: display: fontFamily: "Chakra Petch, system-ui, sans-serif" fontWeight: 500 lineHeight: 1.06 letterSpacing: "-0.025em" prosa: fontFamily: "Chakra Petch, system-ui, sans-serif" fontWeight: 400 lineHeight: 1.75 dado: fontFamily: "Chakra Petch, system-ui, sans-serif" fontFeature: "tabular-nums" chanfro: botao-lp: "12px" card-lp: "18px" botao: "9px" botao-pequeno: "7px" controle: "7px" card: "12px" etiqueta: "6px" spacing: sidebar: "224px" sidebar-recolhida: "64px" gutter-lp-desktop: "64px" gutter-lp-mobile: "24px"

8. Design da interface — Contexto, não certeza

Direção aprovada no protótipo v3 e implementada em 16/10/2026. Este documento substitui as regras visuais do **Instrumento / Chuva de neon**; não é uma camada adicional sobre elas. A comparação histórica está em Antes e depois da interface.

A verdade de produto continua em `dashboard/PRODUCT.md`. Tokens executáveis moram em `dashboard/app/globals.css`; a forma das camadas (canto, placas, fio, fenda e pincelada) mora em `dashboard/app/camadas.css`; a LP tem seu escopo em `dashboard/app/lp.css`. Os valores abaixo documentam o código, não constituem uma segunda configuração.

8.1 DIREÇÃO

A marca tem matéria; a ferramenta tem clareza. A LP combina o monograma FR, camadas com profundidade, tipografia angular e pinceladas diagonais. A dashboard usa a mesma família tipográfica e as mesmas vozes de cor, mas não recebe uma cena atrás dos dados. A fenda do monograma reaparece como pequeno detalhe de identidade, não como recorte que prejudica texto ou alvo de clique.

- Grafite, branco mineral e ouro fosco na chegada.
- Cabeçalhos e navegação em caixa normal; sem telemetria numerada decorativa.
- Números como texto legível, não polígonos de sete segmentos.
- Controles e regiões com **canto cortado** na diagonal da fenda, superfície opaca e borda simples que continua nas duas diagonais. Na LP a peça entra inteira (placas que se abrem); na dashboard, contida (placas só no botão primário).
- Uma região dominante por tela; apoio não disputa o mesmo peso visual.

8.1.1 Camadas do FR

Botões, cards e títulos repetem o monograma: placas empilhadas como a extrusão do FR e o canto cortado da fenda. **Uma fonte só:** o polígono do canto e as cores das placas existem apenas em `app/camadas.css`, em `@layer camadas` (acima dos utilitários, para `rounded-*`, `bg-*` e `outline-none` não apagarem o canto nem o foco). A API

pública são classes e variáveis; nenhuma classe define as variáveis PÚBLICAS (`--chanfro`, `--placa-*`, `--cor-*`), só lê com valor de reserva; as internas (`--camada-*`, `--traco-cor`) são definidas pelas classes.

CLASSE	O QUE DESENHA
`.chanfro`	canto cortado no topo-esquerdo e no inferior-direito, com o traço de 1px da borda nas diagonais (`--cor-borda-chanfro`; elemento sem borda usa `transparent`)
`.placas`	face no `::before`, placa 2 no `::after`, placa 3 no `drop-shadow`; no hover a face sobe e as placas abrem
`.placas-so-hover`	as mesmas placas, planas em repouso
`.fio`	2px de ouro no topo à direita, 40% da largura
`.fio-hover`	sublinhado de 2px que cresce no hover e no foco
`.fenda`	risco diagonal pequeno no canto superior direito
`.pincel`	pincelada inclinada (`skewX(-24deg)`) atrás da palavra-chave
`.chanfro-marco`	o canto no `::before` de um marcador (o ponto do `.step`)

- Fio, fenda e o traço do canto **compõem** por variáveis numa única `background-image`: `superficie fio`, `stat placas fio` `fenda` funcionam juntos. Por isso `.superficie` usa `background-color`, nunca o atalho `background`.
- **Placas são pseudo-elementos, nunca filhos:** `data-embaralhar` reescreve o `textContent` do botão da LP. A camada anula `border`, `ring`, `shadow`, `bg` e `rounded` do próprio elemento, que deve tirar essas classes; a borda da face vem de `--placa-borda`. A transição das placas não inclui `opacity`: o GSAP lê o valor final de `.rv` e o botão nasceria invisível.
- **Foco visível por dentro.** O `clip-path` corta anel externo: em `.chanfro` e `.superficie` o anel é `outline` com deslocamento −2px; em `.placas`, sem clip-path, o anel fica a 6px, longe da face que sobe. CSS sem camada com `outline-offset` positivo vence essa regra; a LP repete o −2px em `.lp .chanfro:focus-visible`.
- `.superficie` tem `clip-path`: conteúdo absoluto que não seja portal e transborde a região é cortado.
- `filter` em `.placas` cria bloco de contenção para `position: fixed`: sobreposição dentro de algo com placas vai por portal.
- **Tokens das placas:** `--placa-2/3` e `--placa-card-2/3` seguem o tema (`#a8904f`, `#6f5f36`, `#34302a`, `#2a2621` no escuro; no claro, derivados dos tokens do claro por `color-mix`, sem hexadecimal do escuro). A LP é sempre escura e lê `--placa-escuro-*`; o fecho dourado lê `--placa-fecho-*` (face grafite `#171819`, placas `#3a3326` e `#7a6532`, texto ouro).
- Ouro continua sendo ação e marca: o fio, a fenda e a pincelada são identidade, não valor.

8.2 COR E SIGNIFICADO

PAPEL	ESCURO	CLARO
Fundo	`#171819`	`#F3F4F5`
Superfície / popover / sidebar	`#222426`	`#FFFFFF`
Texto principal	`#F3F4F5`	`#202326`
Texto secundário	`#B3B9C0`	`#565D65`
Ação e foco	`#D6BA72`	`#735713`
Dito	`#EDB466`	`#8A4B00`
Medido — marcas gráficas	`#8DC8F5`	`#165E9D`
Texto de medidas	`#F3F4F5`	`#202326`
Borda	`#454B52`	`#D5D9DE`

Âmbar é dito; azul identifica as marcas gráficas do medido. Fala, texto e emoji pertencem ao primeiro; probabilidade, nota, tempo e tendência, ao segundo. Por decisão de 22/10/2026, os textos antes azuis usam branco mineral no escuro e grafite no claro (`--medido-texto` acompanha `--foreground`). Na LP, sempre escura, o token acompanha `--texto`. Números, títulos e legendas ficam neutros; séries, bordas e marcadores continuam com suas cores semânticas. Uma régua entre dito e medido só aparece quando existe essa divisão real; NPS × latência são duas medidas, não uma fala confrontada com uma medida.

Ouro não é dado. Pertence a marca, ação, foco e seleção. Não colore série, barra, categoria ou valor de indicador para parecer bom resultado.

Categorias, famílias, emoções e tipos de nó conservam seus significados. No tema claro, a luminância se adapta ao fundo, não se troca a família de matiz. Toda categoria leva rótulo; cor não é o único canal. Os validadores medem texto e marcas nos dois temas, inclusive as cores do grafo.

8.3 TIPOGRAFIA E ÍCONES

Chakra Petch local, pesos 400, 500 e 600, com licença SIL OFL em `dashboard/public/identidade/`. `next/font/local` serve os arquivos do próprio produto; não há busca de fonte no Google no build ou no navegador.

- Texto, títulos, controles e valores usam a família quadrada.
- Valores comparáveis usam algarismos tabulares. Código, `pre` e `kbd` mantêm pilha monoespaçada de sistema.
- Piso de texto: 11px. Não diminuir procedência ou ressalva para fazê-la caber.
- Títulos grandes fecham o tracking; não aplicar caixa alta e tracking de telemetria a toda a interface.
- Lucide com ícones contextuais. Ação usa o desenho do verbo; seleção é identificada pelo estado, não por um ícone arbitrário.
- Ícone decorativo leva `aria-hidden`; texto ou nome acessível diz a ação.

8.4 LP

A LP segue o mockup GSAP + 3D aprovado em 19/10/2026, que **substituiu** as regras de movimento anteriores (palco sticky, troca de pose em 650ms, sem partículas nem laço). O conteúdo continua SSR e completo sem JS; a coreografia inteira mora em `components/lp-nova/Coreografia.tsx`, o único componente cliente.

8.4.1 Estrutura e conteúdo

1. **Abertura:** “O atendimento termina. A análise começa.”, ação para conhecer o método e quatro fatos do motor: 7 famílias, 39 features, 3 BERTimbau, 0 LLM. Vêm de `FATOS_LP`; `tests/test_lp_fatos.py` falha se o fusor divergir. O servidor já entrega o número final; o contador só anima por cima.
2. **Letreiro** “Contexto · não certeza · Dito · Medido”, decorativo (`aria-hidden`).
3. **Contexto:** “Ok, obrigado.” identificado como **exemplo de linguagem**; pedido, resposta e desfecho dizem o que conferir.
4. **Sinais:** leitura da mensagem, contribuição dos sinais e tempos; as leituras gravadas seguem em expansão nativa logo abaixo.
5. **Faixa:** TEXTO, EMOJIS e TEMPO, fixada e rolada na horizontal.
6. **Uso:** panorama → recorte → conversa, com destino real para `/dashboard`.
7. **Limites:** estimativa, não pesquisa; não avalia atendentes; tempo treinado com dados sintéticos; sem sinal não é zero.
8. **Fecho:** o mundo vira ouro; entrar e criar conta.

O rodapé diz “toda **nota** é inferida”: os contadores do topo são fatos do código, não estimativas. `carregarLeituras` valida o snapshot; sem snapshot válido a LP nomeia a ausência e não publica resultado do dublê. Nenhuma análise é executada ao abrir a LP.

8.4.2 Céu

O fundo da LP é o céu da arte do João, em movimento: `<Ceu />` (`.lp-ceu`), fixo atrás de tudo (`z-index: 0`, abaixo do palco WebGL) e sem ponteiro. São 140 pontos dourados (`#e9cf8e`, 8% grandes; 60% piscam em ciclos de 2,5 a 7 s) e 26 riscos finos a −22° da vertical, que descem em 14 a 30 s.

- **Só CSS:** sem canvas e sem script por quadro; anima apenas `opacity` e `transform`/`translate`. As posições saem de um PRNG de semente fixa (`lib/lp-nova/ceu.ts`), renderizadas no servidor: sem `Math.random()` no render, a hidratação bate.
- **Fecho:** com `.lp.inv` o ouro entra por opacidade do `::before` em 0,6 s (gradiente não interpola), os pontos somem e os riscos escurecem, como na arte dourada. Nenhum código da coreografia muda.
- **O fecho dourado só existe com JS e movimento ligados** (`.inv` vem do ScrollTrigger); em reduce ou sem JS o fecho continua escuro.
- **Reduce:** o céu fica parado, com os riscos visíveis e a mesma cara da arte.
- **Sem JS:** o céu é HTML do servidor com animação em CSS; chega completo.

É a **única animação contínua permitida** fora do palco, e só porque é CSS e fica atrás dele. A dashboard não recebe o céu.

8.4.3 Widgets da LP

Botão primário `btn placas` com três placas (face ouro, `#a8904f` e `#6f5f36`), canto de 12px; no hover a face sobe 3px e a placa de trás desce. Botão secundário `btn o chanfro fio-hover`: contorno e fio que cresce embaixo. `.card` e `.stat` têm face com canto de 18px, duas placas que se abrem no hover e fio no topo; a `.stat` leva a fenda no canto e o rótulo reserva espaço para ela. `.ck li`, abas e o resumo das leituras têm canto e contorno, sem placas; `.tag` tem canto de 6px; o `.step` muda só o marcador, um quadrado cortado. A palavra “análise” do título da abertura leva a pincelada, dentro de uma máscara com folga para não sair cortada. No fecho as placas viram grafite com texto ouro. Os espaços de `.stats`, `.grid` e `.card` são de 16px, para as placas abertas não encostarem na vizinha.

8.4.4 Tipografia da LP

Unbounded (títulos, letreiro, preloader) e Space Grotesk (corpo), variáveis, locais em `public/identidade/` com SIL OFL, carregadas por `next/font/local` **só em** `app/page.tsx`: a dashboard não baixa nenhuma delas. Rótulos, botões e números seguem em Chakra Petch.

8.4.5 Coreografia

- **Preloader** dourado com contador, **uma vez por sessão**: o script do `<head>` (`SCRIPT_PRELOADER`) marca `data-lp-pre="visto"` antes do paint; quem volta de `/entrar` não espera de novo.
- **Lenis** no ticker do GSAP, âncoras pelo próprio Lenis. Títulos sobem palavra a palavra (quebradas no SSR, nunca `innerHTML`), revelações em lote, frase letra a letra com o texto inteiro para leitor de tela, régua e linha dos passos por scrub, botões magnéticos, cursor próprio só com `hover: hover`, texto embaralhado com nome acessível fixo, índice lateral `01 / 07`.

- O letreiro só anima dentro da viewport; ao voltar de baixo ele retoma em reverso sem parar no início da timeline. Cursor e magnetismo só registram ponteiro com `hover: hover` e `pointer: fine`.
- **Desmontagem**: cada listener, tween, ticker e o Lenis são revertidos; o QA confere que nada sobra em `/entrar`.

8.4.6 FR e palco

O **modelo é intocável**: contornos de `escultura.ts` (do SVG oficial, com a fenda) e o material do shader. Câmera, poses e coreografia são livres. As cinco placas **não representam** famílias, features ou contribuições do modelo.

- Um canvas WebGL fixo de tela cheia (`lib/lp-nova/palco-fr.ts`): câmera de 35°, dois programas (FR e 700 partículas em disco, PRNG fixo), dois draws por quadro, DPR até 1,5 e no máximo 2,2 megapixels.
- Geometria e shaders carregam no primeiro respiro do navegador, não no bundle inicial da coreografia. O WebP SSR permanece disponível enquanto o palco prepara; formato, material, partículas e poses não foram simplificados.
- Cada quadro reutiliza o mesmo estado numérico. Perder o contexto destrói a instância anterior antes de restaurar; listeners não se acumulam a cada ciclo.
- Cada pose de `POSES_POR_SECAO` chega com a própria seção (`fromTo` com scrub). Uma timeline única sobre o `<main>` desalinhava por causa da faixa.
- Pulso entre placas, giro e letreiro leem a velocidade do Lenis **suavizada** a cada quadro. A velocidade crua chega em degraus pela roda do mouse e fazia o FR pular. Passos usam `deltaRatio`, então 120 Hz não gira em dobro.
- No fecho o FR vira relevo do próprio ouro, um tom abaixo. Grafite apagava o título escuro por cima.
- Celular: o mesmo palco, centralizado, escala 0,55 e opacidade 0,38.
- Reduce: um quadro parado, opacidade 0,3, sem preloader, pin, Lenis ou letreiro.
- Backend de software (SwiftShader, llvmpipe...), contexto recusado ou perdido: `fr-lp-0.webp` (ou a variante compacta) no lugar. O WebP é captura do mesmo modelo; o exportador antigo saiu com a narrativa sticky.

8.5 DASHBOARD

Sidebar de **224px**, recolhida em um rail de **64px**, com ícones e nomes em caixa normal. Dois grupos organizam o dia a dia: **Acompanhar** (visão geral e atendimentos) e **Investigar** (analisar, grafo e modelo, este só para `dev`). Gestão e configuração moram no **menu da conta** (`MenuDoUsuario`, `Menu` do Base UI), aberto pelo nome no rodapé: Operação, Integrações e Configurações (as duas últimas só `dev`), documentação, tema claro/escuro e

sair. Destinos, ícones, grupos e visibilidade têm uma fonte única em `lib/navegacao.ts` (`gruposDoPapel` para a lateral, `telasDaConta` para o menu).

- Página atual usa `aria-current="page"`; sua seção ancestral usa `"location"`. Links recolhidos mantêm nome acessível e tooltip por foco ou ponteiro.
- Somente `de` e `ate` acompanham a troca de tarefa. Busca, paginação e outros parâmetros locais não vazam para a próxima seção.
- Recolhimento é restaurado no servidor pelo cookie de preferência. Ctrl/Meta+B alterna a navegação sem interceptar input, textarea, select ou contenteditable.
- Ação de recolher/expandir fica junto à marca no desktop. No celular, o cabeçalho abre a gaveta; fechamento explícito e Escape devolvem o foco. Navegação comum fecha a gaveta; clique modificado para nova aba não a fecha.
- Marca interativa e link para a visão geral são controles separados. Tema, documentação, sessão e saída permanecem disponíveis; o rodapé compacto não trunca o aviso real de motor sintético.

Permissões continuam sendo contratos do servidor, não consequência de mostrar ou esconder links. Não converter ausência de papel em permissão administrativa.

A visão geral abre com título e filtro, faixa de quatro indicadores e NPS × latência como região dominante. A faixa são **quatro cards separados** (`.superficie`, canto de 12px, espaço entre eles), em quatro colunas em telas largas e duas em larguras menores; o NPS leva o fio dourado. O resumo do atendimento e o esqueleto de carregamento usam o mesmo card, sem fio: uma linguagem só de indicador. Informação de amostra e incerteza continua próxima do NPS, sem transformar ressalva em rodapé invisível.

A dashboard recebe as camadas **contidas**:

- **Canto cortado** em botões (9px; `sm`/`xs` 7px), campos, `select`, `textarea`, gatilhos de abas (7px), badges (6px) e regiões com caixa própria (`.superficie` e `alert`, 12px). `.superficie` já é cortada; o tooltip usa `superficie-flutuante`, e a gaveta móvel da sidebar não leva `.superficie`.
- **Placas só onde se clica:** o botão `default` tem as placas da LP, menores. `outline` e `secondary` têm canto, contorno e fio no hover; `destructive`, só canto (ouro é ação, não alerta). `ghost`, `link` e botões de ícone não têm canto nem placa: num quadrado de 24 a 36px o corte vira ruído. Card que não é clicável não ganha placa, porque a placa avisa “isso é clicável”.
- **Fio dourado** no NPS da faixa e no painel `dominante` (`superficie fio`). O `apoio` continua só com a linha no topo, sem caixa.
- Sobreposições (tooltip, popover, sheet e menus) não mudam. O tema claro usa as mesmas formas com placas derivadas dos tokens do claro.

`Painel` preserva dominante/apoio, erros locais, vazios e aparato metodológico. Superfície opaca, borda simples; nenhum ateliê, brilho especular, vidro ou fundo animado interfere nos gráficos. `EstadoVazio` nomeia o que falta e o próximo passo; o endpoint de origem fica em **Método e ressalvas**, não na face do painel.

8.5.1 Leitura progressiva — 22/10/2026

- **Analisar:** mantém “arrastar e pronto”. Depois da leitura, a confirmação de gravação e o resumo vêm primeiro; a conversa fica legível sem marcações. Probabilidades, sinais por mensagem e contribuições só aparecem quando pedidos. Apenas o primeiro resumo do lote é dominante; os demais são apoio.
- **Vazios confirmados:** sem histórico disponível, orientar ao primeiro arquivo e, com permissão, à integração. Com histórico fora do filtro, oferecer limpar período. Falha de leitura da API não pode virar convite para importar.
- **Período:** datas manuais e atalhos de 7 dias, 30 dias e todo o período compartilham `de`/`ate` na URL. O fim dos atalhos é o último dia com dados, explicitado junto aos controles; não é a data de hoje.
- **Apoio legível:** descrições de indicadores, períodos, vazios, método e drivers usam 14px (`text-sm`) com entrelinha confortável. Eixos e outras marcas densas preservam a escala própria, sem aumentar toda a interface indiscriminadamente.
- **Honestidade não recolhe:** estimativa, sem sinal, limitações do tempo sintético e avisos sobre o arquivo permanecem visíveis. Endpoints, definições técnicas e escala logit entram no aparato existente, sem um segundo sistema de notas explicativas.

8.5.2 Atividade e carregamento — 22/10/2026

- A conferência de colunas carrega apenas quando solicitada; a espera tem `role="status"`. Não adicionar uma segunda chamada de leitura ou inferência para preparar os seletores.
- A sonda de saúde fica suspensa com a aba oculta e consulta ao voltar. Polling e tentativa manual compartilham somente a Promise em curso na mesma instância do provider. Isso não é cache de indicadores, sessão ou credencial; a cadência visível e os estados real/dublê/frio/erro permanecem os mesmos.

8.5.3 Números, ausência e proveniência

`ValorNumerico` recebe uma string já formatada, rótulo e ausência explícita. Não calcula score, nota ou categoria. `NotaInferida` apresenta a nota recebida. Zero observado continua zero; `null` continua ausência.

- NPS e CSAT carregam **estimativa**. Contenção e latência são observadas.
- Amostra insuficiente não ganha ponto de NPS; o intervalo amostral não mede o erro do modelo.
- `sem sinal`, `sem estimativa` e `sem dados` não são sinônimos; a região nomeia a condição correta.
- Categoria medida: cabeça cheia; evidência fraca: tracejada com motivo; ausência: cabeça vazada cinza e texto. Cinza fica fora da escala.
- Erro de uma consulta não apaga indicadores que responderam.
- O segredo da marca nunca modifica indicadores, gráficos ou transcrições.

8.5.4 Encaixe secreto

Cinco cliques consecutivos na marca, com até dois segundos entre eles, abrem um quebra-cabeça do FR. Três fragmentos da arte pré-renderizada deslizam em trilhos nativos; o centro de cada trilho recompõe o monograma. A conclusão é uma brincadeira visual, não score, leitura psicológica ou resultado do modelo.

O componente carrega apenas ao descobrir o segredo. Não usa canvas, áudio, relógio de jogo, API ou persistência; não fecha sozinho. Embaralhar oferece outro arranjo. `prefers-reduced-motion` elimina transições sem retirar o jogo. O diálogo contém foco, fecha por Escape/botão/clique fora e devolve foco à marca. No celular, fechar o segredo não fecha a gaveta que o abriu.

A antiga revelação vaporwave, a pontuação do ritmo e o contexto que falsificava indicadores foram removidos, não mantidos como um segundo modo.

8.5.5 Temas

Escuro é o padrão; **Claro** é alternativa da dashboard. Preferência fica no navegador e acompanha abas e navegação. Valores legados Instrumento/Chuva não são aliases: retornam ao padrão. LP e acesso ficam escuros, sem apagar a preferência da ferramenta; voltar à dashboard restaura a escolha.

O script inicial evita flash; `TemaDaRota` trata transições do App Router. O canvas do grafo invalida a paleta ao trocar tema sem recriar dados ou física.

8.6 ACESSO, ACESSIBILIDADE E ESTADOS

O acesso mantém o FR pré-renderizado de luz de corte: uma passagem curta, sem loop, interrompida ao operar o formulário. Não carrega motor 3D no navegador. Convite, código administrativo, erros, disponibilidade de login e navegação são contratos do servidor, não efeitos visuais.

- Teclado: pular conteúdo, foco visível, abas com setas/Home/End e `details` nativo com Enter. Controles da LP têm alvo mínimo de 44px.
- Carregamento: esqueleto com forma do resultado e estado ocupado, nunca zero ou “sem sinal” antes da resposta.
- Erro: causa visível e ação real, sem ocultar os vizinhos que funcionam.
- Movimento reduzido remove deslocamento; não apenas o torna mais lento.
- Não pontuar atendentes, não alegar leitura de pensamentos e não apresentar números de demonstração como resultados de clientes reais.

8.7 VERIFICAÇÃO E MANUTENÇÃO

`npm test`, `npm run lint`, `npm run build`, `npm run contraste` e `npm run pisos` cobrem contratos diferentes. Nenhum substitui olhar e operar o produto.

`npm run test:lp` exercita sem JS, reduce, preloader por sessão, pin, índice, inversão do fecho, ida e volta para `/entrar` sem sobras, sem WebGL, celular sem rolagem lateral e o redirect de `/leitura`. `LP_EVIDENCIAS` guarda capturas. Sem GPU o Chrome headless usa SwiftShader e o palco cai no WebP, de propósito; `LP_GPU=1` pede GPU real para exercitar o WebGL. O vídeo do acesso derruba a GPU do headless com Vulkan, por isso o QA bloqueia mídia: é artefato do teste.

`DASHBOARD_URL=http://127.0.0.1:3017 npm run test:atividade` verifica pausa de sondas ocultas, reconsulta ao voltar, concorrência com tentativa manual e recuperação da disponibilidade. Só aceita host local; controla visibilidade e falhas HTTP no navegador, sem gravar dados nem provar inferência real. O QA da LP também confere o letreiro ao voltar do fecho e sua pausa fora da tela; com `LP_GPU=1`, inclui ciclos de perda/restauração sem acumular listeners.

`node scripts/validar-navegacao-playwright.mjs` opera rotas reais, período, recolhimento/reload, atalhos durante edição, tooltip por foco, tema, marca e gaveta móvel. `DASHBOARD_URL` aponta para o produto já iniciado; exige sessão válida ou o modo local aberto. Também conferir dashboard vazia, atendimento sem sinal, temas, grafo e navegação LP ↔ ferramenta.

`node scripts/validar-segredo-playwright.mjs` resolve o jogo por teclado, confere mudança dos pixels, reinício, embaralhar, foco e modais aninhados. Compara os quatro indicadores antes/durante/depois em desktop e celular, incluindo Claro e reduce. `SEGREDO_EVIDENCIAS` guarda as capturas opcionais.

A documentação MkDocs mantém sua aparência própria nesta entrega; não presumir que o CSS do site foi redesenhado junto com o produto.

9. LP e dashboard — antes e depois

16/10/2026 · direção FR aprovada no protótipo v3 e aplicada ao produto.

A versão anterior usava o vocabulário do **Instrumento**: orbe, fósforo de osciloscópio, rótulos monoespaçados e números em sete segmentos. A nova direção faz do **FR oficial** a assinatura visual: matéria clara e champanhe sobre grafite, letras quadradas e pinceladas diagonais. Na ferramenta, a expressão da marca fica no chassi; não há uma cena competindo com os dados.

Este registro compara implementação e composição. **Não é um estudo de usabilidade, uma medição de acurácia ou uma prova de publicação em produção.** As regras vigentes estão em Design da interface.

9.1 CONDIÇÕES DA COMPARAÇÃO

- **Antes:** produto exportado da revisão `e5aa19d859e1f5cc402958a16ae43450abc9a905`, sem alterar seu desenho. Capturas em servidor de desenvolvimento; o pequeno indicador circular do Next não faz parte da identidade do produto.
- **Depois:** build de produção local do redesign, na mesma máquina e Chromium. Não houve deploy nesta entrega.
- **Enquadramento equivalente:** desktop de 1440 × 900 e celular de 390 × 844 pixels CSS, escala de captura 1,25. Cada par mostra o mesmo tamanho de viewport.
- **Dados iguais:** snapshot de demonstração versionado, 64 conversas sintéticas, sendo 59 com sinal e cinco sem sinal; período completo de 01/08 a 22/08/2026. O banco foi copiado para uma instância temporária compartilhada pelas duas interfaces, sem reanalisar ou modificar as conversas.
- **Proveniência do snapshot:** SHA-256 `a18a4fa04e0a4ba74f145625727ba6812893327f7cf9c87a22849f08e2fbe9ef`.
- A API de demonstração identifica explicitamente **motor dublê — números sintéticos**. Esses números servem para conferir a interface, não são resultados de clientes reais nem validação do classificador.
- A LP estava **sem snapshot válido de leituras gravadas**. As duas versões mantêm ausência explícita; nenhum resultado foi criado para preencher a tela.

Clique nas imagens para abrir a captura em tamanho original.

9.2 LANDING PAGE


Fraus
Um obrigac
não conta t
ok, obrigado 🙂
CASO SINTÉTICO · COBRANÇA SEM SOLUÇÃO
O Fraus estima satisfação em atendimentos de chatbot, s
pesquisa. Inspecione o contexto antes de agir.
Entender como funciona
Entrar